UNIVERSITY OF
SOUTH DAKOTA
BEACOM SCHOOL OF BUSINESS

DSCI 723

Data Management and Warehousing

Database Design for Canyon Ranch

Group 5 – Milestone 1

Group Members:

Sanjay Kumar Singh
Abigail Torome
Swetha Lnu
Isaac Wainaina
Sharon Wamalwa

Supervised Under:
Bartlomiej Hanus

Semester - Spring 2026

15th March 2026

Beacom School of Business

University of South Dakota

Vermillion, SD, USA

Abstract

Canyon Ranch is a luxury health and wellness resort operating across five properties and offering more than 500 integrated spa, health, and hospitality services. Despite its strong brand reputation, the organization faces significant data management challenges stemming from fragmented information systems, including the legacy Computerized Lodging Systems (CLS), the standalone Guestware application, paper-based activity tracking, and the absence of integrated point-of-sale systems. These disconnected systems limit the organization's ability to access comprehensive guest profiles, analyze service utilization patterns, and support data-driven decision making. As competitive pressure increases in the wellness industry, Canyon Ranch requires a unified and structured data environment capable of supporting both operational efficiency and strategic analytics.

Milestone 1 addresses this business problem through the development of a rigorous conceptual data model using Entity-Relationship (ER) modeling techniques. The team translated the narrative case requirements into a structured ER diagram that identifies core operational entities such as Guest, Stay, Property, Room, Staff, Service, Package, Transaction, Product, and Activity. Weak entities, including CentennialMembership and ServiceVoucher, were introduced where existence depends on a parent entity, while associative entities such as ServiceAppointment, StaffQualification, Transaction, and ActivityAttendance were designed to resolve many-to-many relationships. The Stay entity serves as the central anchor of the model, integrating guest reservations, service bookings, financial transactions, and activity participation.

The key deliverables for this milestone include a clearly defined business problem statement, a complete ER diagram produced in ERDPlus, explicit specification of entity attributes and primary keys, correct relationship cardinalities with participation constraints, and a written ER narrative justifying major design decisions. Together, these deliverables establish a normalized conceptual foundation for subsequent logical, physical, and data warehouse design phases while directly addressing Canyon Ranch's need for integrated guest intelligence and improved operational visibility.

Table of Contents

Abstract 2

1. Problem Statement 13

2. ER-Diagram 16

3. Mapping Requirements 17

3.1 Guest and Guest Profiles 17

3.2 Property and Business Units 18

3.3 Service Catalog 19

3.4 Staff and Service Providers 20

3.5 Reservations, Stays, and Service Bookings 20

3.6 Packages 21

3.7 Charge and Transaction 21

3.8 Lectures, Fitness Classes and Outdoor Activities 22

4. ER-Narrative 22

5. Contribution Statement 25

References 28

ABSTRACT – Milestone 2 29

MILESTONE 1 REVISION SUMMARY 35

RELATIONAL SCHEMA 36

REGULAR ENTITIES 36

STAFF 36

PGCOORDINATOR 36

PROPERTY 37

ROOM 37

DEPARTMENT 37

PACKAGE 37

SERVICECATEGORY 37

SERVICE 38

DELIVERS 38

RESERVATION 38

GUEST 38

STAY 39

ACTIVITY 39

ACTIVITYATTENDANCE 39
PRODUCTS 40
GUESTFOLIO 40
TRANSACTION 40
QUESTIONNAIRE 40
WEAK ENTITIES 41
CENTMEMBERSHIP 41
SERVICEVOUCHER 41
MULTI-VALUED ATTRIBUTE MAPPING 41
GUEST_PHONENO 42
SERVICECATEGORY_SUBCATEGORY 42
M:N Relationship Mapping 42
PAYS 42
HEALTHDATA 42
FUNCTIONAL DEPENDENCY ANALYSIS AND NORMALIZATION 43
STAFF 43
PROPERTY and ROOM 44
ROOM(RoomID, RoomNo, RoomType, Capacity, BaseRate, PropertyID) 44
SERVICE and SERVICECATEGORY 44
SERVICE 45
GUEST 45
STAY 46
ACTIVITY 46
ACTIVITYATTENDANCE 47
CENTMEMBERSHIP (Weak Entity) 47
GUEST_PHONENO (Multi-valued attribute relation) 48
GUESTFOLIO 48
PHYSICAL DATABASE IMPLEMENTATION IN SQL SERVER 49
SQL Server Environment 49
Table Implementation and Constraints 50
Referential Integrity and ON DELETE / ON UPDATE Decisions 50
Indexing Strategy 51
Submission Artifacts 51

SAMPLE DATA POPULATION 52
Data Generation Approach 52
Domain-Specific Rules Applied 52
Core Table Row Requirement Compliance 53
Submission Artifacts 53
Advanced SQL Queries 54
Query 1 (Multi-Table JOIN): 54
Query 2 (Aggregate + GROUP BY / HAVING): 55
Query 3 (Subquery or CTE): 56
Query 4 (Window Function): 58
Explanation: 62
Query 5 (Correlated Subquery): 63
Query 6 (DML Transaction): 66
VIEWS 70
View 1 – Operational 70
(a) Business Purpose 70
(b) CREATE VIEW Statement 71
Interpretation of Output (View 1 – vw_GuestStayDetail) 72
VIEW 2 ANALYTICAL 73
vw_ServiceUtilizationByHour 73
(a) Business Purpose 73
(b) CREATE VIEW Statement 73
Interpretation of Output (View 2 – vw_ServiceUtilizationByHour) 75
Contribution Statement – Milestone 2 76
GUEST.ReservationID FK is backwards 80
ACTIVITY.StayID FK makes activities stay-specific 80
ACTIVITYATTENDANCE is 1:1 with ACTIVITY 80
No DELIVERS bridge table 80
Stored Procedure — Missing Validation 81
Query Mismatches — Three Queries Need Correction 82
Query 1 82
Query 2 83
Query 3 84

Query 4 - Self Awareness is Good ..... **Error! Bookmark not defined.**
Views 86
Milestone 3 Abstract 88
DIMENSIONAL DATA WAREHOUSE CONCEPTUAL DESIGN 89
1. Schema Selection and Justification 89
2. Fact Table Grain Definition 89
3. Slowly Changing Dimension (SCD) – Guest 90
4. Data Warehouse Schema Diagram Explanation 91
DW PHYSICAL IMPLEMENTATION 91
DW SPECIFIC QUERIES 92
Explanation 94
Explanation 97
Explanation 98
Explanation 99
DATA SENSITIVITY CLASSIFICATION 99
Tier 1 : Public 100
Tier 2 : Internal 100
Tier 3 : Confidential 101
Tier 4 : Restricted / PHI (Protected Health Information) 101
FIVE PRIVACY RISK SCENARIOS 102
Privacy Risk Scenarios 102
Risk 1 Unauthorized Access to Health and Medical Data 102
Threat Scenario 102
Potential Harm 102
Recommended Mitigation 103
Risk 2 Guest Profile Re-identification Through Combined Data 103
Threat Scenario 103
Potential Harm: 103
Recommended Mitigation 104
Risk 3 Insecure Storage of Financial and Payment Data 104
Threat Scenario 104
Potential Harm 104
Recommended Mitigation 105

Risk 4 Inadequate Access Controls on Guest Preference and Behavioral Health Data 105
Threat Scenario 105
Potential Harm 105
Recommended Mitigation 106
Risk 5 Data Retention and Right to Erasure Non-Compliance 106
Threat Scenario 106
Potential Harm 106
Recommended Mitigation 107
REGULATORY ANALYSIS (HIPAA, CCPA, GDPR) 107
Regulatory Environment 107
HIPAA : Health Insurance Portability and Accountability Act 108
CCPA / CPRA : California Consumer Privacy Act and California Privacy Rights Act 109
GDPR : General Data Protection Regulation 110
ETHICAL DESIGN DECISIONS 111
Ethical Design Decision 1 112
Storing MedicalAlertFlag in the SERVICEAPPOINTMENT Table 112
Ethical Design Decision 2 113
Accessibility of Guest Preference Data Across Staff Roles 113
GOVERNANCE RECOMMENDATIONS 115
Recommendation 1 115
Implement Role-Based Access Control (RBAC) Across All Database Schemas 115
Recommendation 2 116
Implement Audit Logging via Database Triggers on Sensitive Tables 116

5. Contribution Statement 16

Table of Figure:

Abstract 2
1. Problem Statement 13
2. ER-Diagram 16
3. Mapping Requirements 17
3.1 Guest and Guest Profiles 17
3.2 Property and Business Units 18
3.3 Service Catalog 19
3.4 Staff and Service Providers 20
3.5 Reservations, Stays, and Service Bookings 20
3.6 Packages 21
3.7 Charge and Transaction 21
3.8 Lectures, Fitness Classes and Outdoor Activities 22
4. ER-Narrative 22
5. Contribution Statement 25
References 28
ABSTRACT – Milestone 2 29
MILESTONE 1 REVISION SUMMARY 35
RELATIONAL SCHEMA 36
REGULAR ENTITIES 36
STAFF 36
PGCOORDINATOR 36
PROPERTY 37
ROOM 37
DEPARTMENT 37
PACKAGE 37
SERVICECATEGORY 37

SERVICE 38
DELIVERS 38
RESERVATION 38
GUEST 38
STAY 39
ACTIVITY 39
ACTIVITYATTENDANCE 39
PRODUCTS 40
GUESTFOLIO 40
TRANSACTION 40
QUESTIONNAIRE 40
WEAK ENTITIES 41
CENTMEMBERSHIP 41
SERVICEVOUCHER 41
MULTI-VALUED ATTRIBUTE MAPPING 41
GUEST_PHONENO 42
SERVICECATEGORY_SUBCATEGORY 42
M:N Relationship Mapping 42
PAYS 42
HEALTHDATA 42
FUNCTIONAL DEPENDENCY ANALYSIS AND NORMALIZATION 43
STAFF 43
PROPERTY and ROOM 44
ROOM(RoomID, RoomNo, RoomType, Capacity, BaseRate, PropertyID) 44
SERVICE and SERVICECATEGORY 44
SERVICE 45
GUEST 45
STAY 46
ACTIVITY 46
ACTIVITYATTENDANCE 47
CENTMEMBERSHIP (Weak Entity) 47
GUEST_PHONENO (Multi-valued attribute relation) 48
GUESTFOLIO 48

PHYSICAL DATABASE IMPLEMENTATION IN SQL SERVER 49
SQL Server Environment 49
Table Implementation and Constraints 50
Referential Integrity and ON DELETE / ON UPDATE Decisions 50
Indexing Strategy 51
Submission Artifacts 51
SAMPLE DATA POPULATION 52
Data Generation Approach 52
Domain-Specific Rules Applied 52
Core Table Row Requirement Compliance 53
Submission Artifacts 53
Advanced SQL Queries 54
Query 1 (Multi-Table JOIN): 54
Query 2 (Aggregate + GROUP BY / HAVING): 55
Query 3 (Subquery or CTE): 56
Query 4 (Window Function): 58
Explanation: 62
Query 5 (Correlated Subquery): 63
Query 6 (DML Transaction): 66
VIEWS 70
View 1 – Operational 70
(a) Business Purpose 70
(b) CREATE VIEW Statement 71
Interpretation of Output (View 1 – vw_GuestStayDetail) 72
VIEW 2 ANALYTICAL 73
vw_ServiceUtilizationByHour 73
(a) Business Purpose 73
(b) CREATE VIEW Statement 73
Interpretation of Output (View 2 – vw_ServiceUtilizationByHour) 75
Contribution Statement – Milestone 2 76
GUEST.ReservationID FK is backwards 80
ACTIVITY.StayID FK makes activities stay-specific 80
ACTIVITYATTENDANCE is 1:1 with ACTIVITY 80

No DELIVERS bridge table 80
Stored Procedure — Missing Validation 81
Query Mismatches — Three Queries Need Correction 82
Query 1 82
Query 2 83
Query 3 84
Query 4 - Self Awareness is Good ..... **Error! Bookmark not defined.**
Views 86
Milestone 3 Abstract 88
DIMENSIONAL DATA WAREHOUSE CONCEPTUAL DESIGN 89
1. Schema Selection and Justification 89
2. Fact Table Grain Definition 89
3. Slowly Changing Dimension (SCD) – Guest 90
4. Data Warehouse Schema Diagram Explanation 91
DW PHYSICAL IMPLEMENTATION 91
DW SPECIFIC QUERIES 92
Explanation 94
Explanation 97
Explanation 98
Explanation 99
DATA SENSITIVITY CLASSIFICATION 99
Tier 1 : Public 100
Tier 2 : Internal 100
Tier 3 : Confidential 101
Tier 4 : Restricted / PHI (Protected Health Information) 101
FIVE PRIVACY RISK SCENARIOS 102
Privacy Risk Scenarios 102
Risk 1 Unauthorized Access to Health and Medical Data 102
Threat Scenario 102
Potential Harm 102
Recommended Mitigation 103
Risk 2 Guest Profile Re-identification Through Combined Data 103
Threat Scenario 103

Potential Harm: 103
Recommended Mitigation 104
Risk 3 Insecure Storage of Financial and Payment Data 104
Threat Scenario 104
Potential Harm 104
Recommended Mitigation 105
Risk 4 Inadequate Access Controls on Guest Preference and Behavioral Health Data 105
Threat Scenario 105
Potential Harm 105
Recommended Mitigation 106
Risk 5 Data Retention and Right to Erasure Non-Compliance 106
Threat Scenario 106
Potential Harm 106
Recommended Mitigation 107
REGULATORY ANALYSIS (HIPAA, CCPA, GDPR) 107
Regulatory Environment 107
HIPAA : Health Insurance Portability and Accountability Act 108
CCPA / CPRA : California Consumer Privacy Act and California Privacy Rights Act 109
GDPR : General Data Protection Regulation 110
ETHICAL DESIGN DECISIONS 111
Ethical Design Decision 1 112
Storing MedicalAlertFlag in the SERVICEAPPOINTMENT Table 112
Ethical Design Decision 2 113
Accessibility of Guest Preference Data Across Staff Roles 113
GOVERNANCE RECOMMENDATIONS 115
Recommendation 1 115
Implement Role-Based Access Control (RBAC) Across All Database Schemas 115
Recommendation 2 116
Implement Audit Logging via Database Triggers on Sensitive Tables 116

1. Problem Statement

Canyon Ranch, founded in 1979, is a luxury health and wellness resort operating in five locations and provides over 500 health and wellness services (Austin et al., 2008). The ranch relies on guest data, staff coordination, scheduling services, and transactional systems to maintain its reputation and provide services to its clients. However, the current system is outdated and does not provide sufficient decision-making data useful in maintaining competitiveness.[1] The ranch maintains independent systems for the resort and SpaClubs, with the destination resorts utilizing Computerized Lodging Systems (CLS). Canyon utilizes source code to CLS which it owns for customization and partners with Computerized Lodging Systems for robust and complex scheduling components for Health and Healing and Spa services (Austin et al., 2008). Most activities are not included in the CLS which focuses on transactional processes rather than collecting decision-making data.[2][3]

The ranch maintains paper-based records for popular services such as signing up for activities. It records and keeps customers data including their schedule, when it happens, people involved and transactional data as well as specific information such as anniversaries and important dates in free-form text that is not easily understood by automated software (Austin et al., 2008). In Berkshires, which is one of the locations, the ranch has a Guestware software application for collecting guest surveying, incident tracking, preference collection and support; however, the software does not integrate well with the CLS (Austin et al., 2008). Additionally, the ranch does not have point-of-sale software for salons and restaurants to enable tracking of purchases independently. With such a system, Canyon ranch requires a well-structured analytical and relational database that will enable it to collect structured data useful for day-to-day operations as well as data analysis.

The first benefit of a well-structured database model is enhanced operational efficiency and inventory management. SQL server relational database system can handle millions of daily requests, efficient for data search, storage, and retrieval (Mohammed, 2025). With an integrated system, the ranch will improve its scheduling, transactions, reservations, and staff assignment across various services and locations. Such improvement will help reduce time used to retrieve and compare data, making decision making easy for the executives, staff and clients (Troisi et al., 2023).

The second benefit of the model is improved guest personalization which improves guest retention. According to Celestin et al., seamless delivery of personalized experience increases customer satisfaction by 30% and customer retention by 25% (2024). With such a system, staff will access guest and staff information at a central place and in a structure that is easy to analyze and make decisions.[4] With data on specific events and dates, preferences, service history and health information being available, it becomes effective for program coordinators to efficiently resolve and overcome conflict on clients' scheduling requests.

The third benefit is competitive advantage and better decision making. Centralized and well-structured data enables data analysis, forecasting of future developments and informed decisions making (Abdulrahman et al., 2022). A study by Hosen et al., shows analytics and reporting using advanced database management systems increases customer retention by 30%, profit margin by 15% and reduces data processing time by 40% (2024). With such advantages, the ranch will be able to achieve competitive advantage in the industry.[5][6]

The implementation of a new database system also comes with some underlying risks. First, the data management system is at risk of cyber-attack which may result in data privacy and confidentiality issues (Farook et al., 2025). Access and misuse of sensitive information by

unauthorized parties during the transition and while operating may raise ethical and legal concerns (Farook et al., 2025). Second, data may be lost, corrupted and inconsistent due to lack of proper cleansing which may hinder efficiency (Gangula, 2024). Third, operational disruption may slow service delivery and efficiency where there is staff resistance in adapting to the new system and limited training (Gangula, 2024).[7]

2. ER-Diagram

The ER diagram is inserted here in exported .png format. Details ERD file is attached with the assignment in Dropbox.

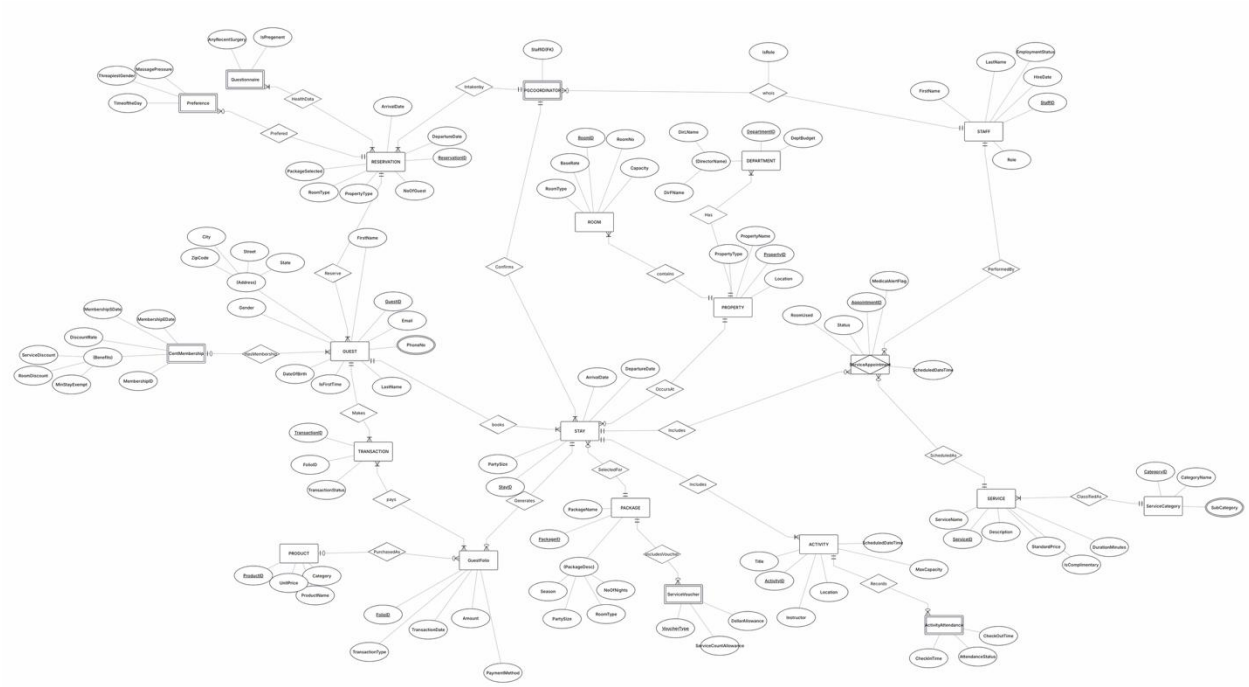


Figure 1: Entity-Relationship Diagram for Canyon Ranch Reservation System

3. Mapping Requirements

3.1 Guest and Guest Profiles

Guest:

- GuestID (PK)
- FirstName
- LastName
- phone number
- email address
- home address
- Date of birth
- Gender (male, female)
- Number of times visited (integer)

Guest Preference:

- Service category
- Therapist gender preference (male, female)
- Time-of-day (morning, midday, evening)

Guest Health Questionnaire:

- Medical alert (pregnancy, recent surgery, etc)

CentennialMembership (Weak Entity)

• MembershipID (Partial Key)

• MembershipDate

• Benefits

• DiscountRate

• ServiceDiscount

• RoomDiscount

• MinStayExempt

Relationship:

• A Guest may have zero or one CentennialMembership.

• A CentennialMembership must be associated with exactly one Guest.

3.2 Property and Business Units

Property:

- PropertyID (PK)
- PropertyName (1. Canyon Ranch, Tucson, Arizona, 2. Canyon Ranch in the Berkshires, Lenox, Massachusetts, 3. The Venetian, Las Vegas, Nevada, 4. The Gaylord Palms, Kissimmee, Florida, 5. The Queen Mary II- Cunard
- PropertyType (Destination/ Health Resort, SpaClub)
- Location (1. Tucson, Arizona, 2. Lenox, Massachusetts, 3. Las Vegas Nevada, 5. Kissimmee, Florida)

Departments:

- DepartmentID

- Directors
- Budgets

Destination resort room:

- RoomID
- Room number
- Season of the Year (19 Oct – 29 Nov, 30 Nov -18 March (excluding 24-31 Dec and 13-15 Feb), 19 March -June 30)
- Room type (Deluxe, Luxury Suite)
- Rate (by type and season)
- Capacity

3.3 Service Catalog

Services

- ServiceID
- ServiceName
- Description
- Duration in minutes
- Standard price
- CategoryID (FK)
- DepartmentID (FK)
- Requires Qualification (Yes, No)

Service Category

- CategoryID

- Service category (1. Health and Healing, 2. Spa and sports services, 4. Complimentary services)
- Subcategory

Service Appointment

- AppointmentID
- ScheduleDate Time
- Status
- RoomUsed
- MedicalAlertCode (optional FK)

3.4 Staff and Service Providers

Staff

- StaffID
- Name
- Role
- Date Hired
- Employment status (Full-time, Part-time, On call)
- Services providers (Therapist, physician, trainers, coordinator)
- Program coordinators (unique scheduling and counseling role)

3.5 Reservations, Stays, and Service Bookings

Guest reservation

- ReservationID
- Arrival date

- Departure Date
- Package selected
- Property
- Room
- Number of guests in the party

3.6 Packages

Package description

- PackageID
- Name (Deluxe, luxury suite)
- Room type (Single, Double, Tripple, Quad)
- Season Year (1. 19 Oct – 29 Nov, 2. 30 Nov -18 March (excluding 24-31 Dec and 13-15 Feb), 3. 19 March -June 30)
- Number of nights (3, 4, 5, 6, 7)
- Party size
- Service allowance per department

3.7 Charge and Transaction

Guest folio

- FolioID
- Package rate
- à la carte service fees
- Products purchases (Beauty products, CDs, books, skin care)
- Service charges or taxes (optional)
- Guest payment (credit card, check, corporate billing)

3.8 Lectures, Fitness Classes and Outdoor Activities

Activities

- Category: (Complementary lecture, Fitness classes, outdoor activities)
- Title
- Instructor
- Scheduled time
- Location
- Maximum capacity
- Attendance

4. ER-Narrative

The conceptual data model for Canyon Ranch is designed to integrate guest information, reservations, services, operational activities, and financial transactions into a unified and structured system that supports both daily operations and strategic decision-making.

A Guest is uniquely identified by GuestID and stores personal and contact information including first name, last name, phone number, email, address, gender, and date of birth. Each guest may optionally enroll in a CentennialMembership, which captures membership specific details such as membership date, benefits, discount rates, and service privileges. Since a membership cannot exist independently without a guest, CentennialMembership is modeled as a weak entity with a partial key and total participation in its identifying relationship with Guest. A guest may have zero or one membership, while each membership must be associated with exactly one guest.

Guests may express their service preferences through the GuestPreference entity, which includes preferred service categories, therapist gender preference, and preferred time of day. Additionally, guests provide medical and health-related information through the GuestHealthQuestionnaire, which records medical alerts such as pregnancy or recent surgery. These entities ensure that services are both personalized and safely delivered.

Guests make Reservations, identified by ReservationID, which include arrival and departure dates, selected package, assigned property, room, and the number of guests in the party. Each reservation is associated with one guest and is linked to a specific property and room. A reservation leads to a Stay, which represents the actual visit and acts as a central entity connecting guest activities, service usage, and financial records.

Each stay takes place at a Property, identified by PropertyID, which stores information such as property name, type, and location. A property consists of one or more Departments, each responsible for managing specific service areas and operational budgets. The accommodation structure is represented by the Room entity, which belongs to a property and includes attributes such as room number, room type, seasonal rates, and capacity. A stay is assigned to exactly one room within a property.

Canyon Ranch offers a wide range of services represented by the Service entity, which includes attributes such as service name, description, duration, and standard price. Services are categorized under ServiceCategory, which organizes offerings into broader categories and subcategories such as health and healing or spa services. Each service is associated with a department and may require specific staff qualifications.

Guests can schedule services through ServiceAppointment, which records appointment date and time, status, assigned room, and any applicable medical alert codes. Each service appointment is linked to a specific service and is performed by a member of the Staff, identified by StaffID. Staff members include therapists, physicians, trainers, and program coordinators, and their attributes include role, employment status, and hire date.

During their stay, guests may participate in various Activities, such as fitness classes, lectures, and outdoor programs. Activities are defined by attributes such as category, title, instructor, scheduled time, location, and maximum capacity. Participation in activities is tracked through attendance relationships that record guest involvement in scheduled sessions.

Guests select a Package, identified by PackageID, which determines the structure of their stay, including room type, duration, season, party size, and service allowances per department. Packages help standardize offerings while allowing flexibility in guest experiences.

Financial transactions associated with a guest's stay are recorded in the GuestFolio, identified by FolioID, which aggregates charges such as package rates, à la carte service fees, product purchases, and applicable taxes. Payments are recorded using different payment methods such as credit card, check, or corporate billing. Products purchased during the stay are also tracked as part of the overall transaction structure.

Overall, the Stay entity serves as the central integration point of the model, linking guests, reservations, services, activities, and financial transactions. This design ensures a normalized, scalable, and efficient data structure that enables Canyon Ranch to improve operational efficiency, deliver personalized guest experiences, and support advanced analytics and decision-making.

5. Contribution Statement

Team Member	Name	Key Contributions	Contributions
Member 1	Sanjay Singh	Schedules & facilitates meetings Maintains shared task tracker with deadlines Compiles final Word document with consistent formatting & citations Collects signatures for Team Contribution Statement Submits final file to D2L	Scheduled and conducted meetings on time. Maintained the shared task tracker with deadlines. Provided ideas and contributed actively to the final ERD. Compiled the final Word document with consistent formatting and citations. Collected signatures for the Team Contribution Statement. Submitted the final file to D2L.
Member 2	Abigail Torome	Reads and annotates Canyon Ranch case Writes 400–600 word Business Problem Statement (identifies core problem, 3+ benefits, 3+ risks) Maps requirements to data domains (traceability list)	Read and annotated the Canyon Ranch case. Wrote a 400–600 word Business Problem Statement, identifying the core problem, 3+ benefits, and 3+ risks. Mapped requirements to data domains (created a traceability list).

		Writes ER Narrative (300–500 words)	Wrote an ER Narrative of 300–500 words.
Member 3	Swetha Lnu	Designs entities & keys for Guest, Reservation/Stay, Room, Staff/Service Provider, Program Coordinator, Centennial Membership Specifies attribute types and justifies PKs Defines all relationships with cardinality & participation Builds & maintains ERDPlus diagram	Designed entities and keys for Guest, Reservation/Stay, Room, Staff/Service Provider, Program Coordinator, and Centennial Membership. Specified attribute types and justified all primary keys (PKs). Defined all relationships with proper cardinality and participation. Built and maintained the ERDPlus diagram.
Member 4	Isaac Wainaina	Designs entities & keys for Service, Service Category, Service Appointment, Package, Service Voucher, Transaction/Charge, Product, Lecture/Class/Activity Specifies attribute types and justifies PKs Defines all relationships with	Designs entities and keys for: Service, Service Category, Service Appointment, Package, Service Voucher, Transaction/Charge, Product, Lecture/Class/Activity. Specifies attribute types and justifies primary keys (PKs). Defines all relationships with

		cardinality & participation - Builds & maintains ERDPlus diagram	cardinality and participation. Builds and maintains the ERDPlus diagram.
Member 5	Sharon Wamalwa	Merges sub-diagrams into integrated ERDPlus diagram Audits relationships for cardinality & participation, documenting justification Checks design complexity (redundancy, over-normalization, missing M:N junctions) Produces final exported diagram image	Merged sub-diagrams into a single integrated ERDPlus diagram. Audited all relationships for correct cardinality and participation, documenting justifications. Checked design for complexity issues, including redundancy, over-normalization, and missing M:N junctions. Produced the final exported diagram image for the Word document.

References

1. Celestin, M., Vasuki, M., Sujatha, S., & Kumar, A. D. (2024). How businesses create personalized experiences to boost customer retention: The role of technology and human interactions in customer satisfaction. *International Journal of Applied and Advanced Scientific Research*, *9*(2), 75-80. DOI: https://doi.org/10.5281/zenodo.13871854
2. Troisi, O., Visvizi, A., & Grimaldi, M. (2023). Digitalizing business models in hospitality ecosystems: toward data-driven innovation. *European Journal of Innovation Management*, *26*(7), 242-277. DOI 10.1108/EJIM-09-2022-0540
3. Al-Surmi, A., Bashiri, M., & Koliousis, I. (2022). AI based decision making: combining strategies to improve operational performance. *International Journal of Production Research*, *60*(14), 4464-4486. https://doi.org/10.1080/00207543.2021.1966540
4. Mohammed, M. A. F. (2025). Integrated IT Inventory and Database System for Efficient Asset Management. *QALAAI ZANIST SCIENTIFIC JOURNAL*, *10*(4), 1368-1393. Doi: 10.25212/lfu.qzj.10.4.54
5. Hosen, M. S., Islam, R., Naeem, Z., Folorunso, E. O., Chu, T. S., Al Mamun, M. A., & Orunbon, N. O. (2024). Data-driven decision making: Advanced database systems for business intelligence. *Nanotechnology Perceptions*, *20*(3), 687-704.
6. Farooq, M., Younas, R. M. F., Qureshi, J. N., Haider, A., & Nasim, F. (2025). Cyber security risks in DBMS: Strategies to mitigate data security threats: A systematic review. *Spectrum of engineering sciences*, 268-290.
7. Austin, Robert D., Lynda M. Applegate, and Deborah Soule. Corporate Information Strategy and Management: Text and Cases. 8th ed. McGraw-Hill, 2008.

MILESTONE 2

ABSTRACT – Milestone 2

Database Design and Implementation for Canyon Ranch

Milestone 2 advances the Canyon Ranch database project from conceptual modeling to full logical and physical implementation in Microsoft SQL Server. The primary business problem addressed is the lack of an integrated, normalized, and enforceable database structure capable of supporting reservations, guest management, service scheduling, billing, wellness activities, and operational reporting within a unified system. To resolve these limitations, the approved Entity-Relationship model from Milestone 1 was transformed into a fully normalized relational schema using the ER-to-relational mapping rules from Jukic et al. (2026).

The revised ER diagram incorporated instructor feedback, including removal of redundant attributes, correction of weak entity modeling, and refinement of relationship constraints. The relational schema was then derived using standard notation and validated through comprehensive functional dependency analysis. All relations were evaluated for partial and transitive dependencies, and compliance with Third Normal Form (3NF) and, where applicable, Boyce–Codd Normal Form (BCNF) was demonstrated.

The normalized schema was implemented in SQL Server using structured CREATE TABLE statements with primary keys, foreign keys, CHECK constraints, DEFAULT constraints, and explicit ON DELETE and ON UPDATE actions aligned with business logic. Performance optimization was supported through non-clustered indexes on high-frequency query columns. The database was populated with domain-consistent sample data (minimum 15 rows per core table), ensuring referential integrity and realistic operational scenarios.

Advanced SQL queries were developed to support analytical and operational decision-making, including multi-table joins, aggregate revenue analysis, Common Table Expressions (CTEs), window functions, and a stored procedure with validation logic and RAISERROR handling. Two views were implemented: an operational guest stay summary for program coordinators and an analytical service utilization view for management demand forecasting.

The final deliverables include a fully executable SQL implementation, normalized schema documentation, query demonstrations with screenshots, and a validated database backup file.

Table of Contents

Abstract **2**
1. Problem Statement **4**
2. ER-Diagram **7**
3. Mapping Requirements **8**
3.1 Guest and Guest Profiles 8
3.2 Property and Business Units 9
3.3 Service Catalog 10
3.4 Staff and Service Providers 11
3.5 Reservations, Stays, and Service Bookings 11
3.6 Packages 12
3.7 Charge and Transaction 12
3.8 Lectures, Fitness Classes and Outdoor Activities 13
4. ER-Narrative **13**
5. Contribution Statement **16**
References 18
ABSTRACT – MILESTONE 2 ..... **19**
MILESTONE 1 REVISION SUMMARY **23**
RELATIONAL SCHEMA **24**
REGULAR ENTITIES 24
STAFF *24*
PGCOORDINATOR *24*
PROPERTY *25*
ROOM *25*
DEPARTMENT *25*
PACKAGE *25*
SERVICECATEGORY *25*
SERVICE *25*
DELIVERS 26
RESERVATION *26*
GUEST *26*
STAY *27*
ACTIVITY *27*
ACTIVITYATTENDANCE *27*
PRODUCTS *28*
GUESTFOLIO *28*
TRANSACTION *28*
QUESTIONNAIRE *28*

WEAK ENTITIES 28
CENTMEMBERSHIP *29*
SERVICEVOUCHER *29*
MULTI-VALUED ATTRIBUTE MAPPING 29
GUEST_PHONENO *29*
SERVICECATEGORY_SUBCATEGORY *30*
M:N RELATIONSHIP MAPPING 30
PAYS *30*
HEALTHDATA *30*
FUNCTIONAL DEPENDENCY ANALYSIS AND NORMALIZATION 31
STAFF *31*
PROPERTY and ROOM ***32***
ROOM(RoomID, RoomNo, RoomType, Capacity, BaseRate, PropertyID) *32*
SERVICE and SERVICECATEGORY *32*
SERVICE *33*
GUEST *33*
STAY *34*
ACTIVITY *34*
ACTIVITYATTENDANCE *35*
CENTMEMBERSHIP (Weak Entity) *35*
GUEST_PHONENO (Multi-valued attribute relation) *36*
GUESTFOLIO *36*
PHYSICAL DATABASE IMPLEMENTATION IN SQL SERVER 37
SQL SERVER ENVIRONMENT 37
TABLE IMPLEMENTATION AND CONSTRAINTS 37
REFERENTIAL INTEGRITY AND ON DELETE / ON UPDATE DECISIONS 38
INDEXING STRATEGY 39
SUBMISSION ARTIFACTS 39
SAMPLE DATA POPULATION 39
DATA GENERATION APPROACH 39
DOMAIN-SPECIFIC RULES APPLIED 40
CORE TABLE ROW REQUIREMENT COMPLIANCE 40
SUBMISSION ARTIFACTS 41
ADVANCED SQL QUERIES 42
QUERY 1 (MULTI-TABLE JOIN): 42
QUERY 2 (AGGREGATE + GROUP BY / HAVING): 43
QUERY 3 (SUBQUERY OR CTE): 44
QUERY 4 (WINDOW FUNCTION): 45

Explanation: ***49***
QUERY 5 (CORRELATED SUBQUERY): 50
QUERY 6 (DML TRANSACTION): 52
VIEWS **56**
VIEW 1 – OPERATIONAL 56
(a) Business Purpose *56*
(b) CREATE VIEW Statement *57*
Interpretation of Output (View 1 – vw_GuestStayDetail) *58*
VIEW 2 ANALYTICAL 59
VW_SERVICEUTILIZATIONBYHOUR 59
(a) Business Purpose *59*
(b) CREATE VIEW Statement *59*
Interpretation of Output (View 2 – vw_ServiceUtilizationByHour) *61*
CONTRIBUTION STATEMENT – MILESTONE 2 **62**

List of Figures:

Abstract **2**
1. Problem Statement **4**
2. ER-Diagram **7**
3. Mapping Requirements **8**
3.1 Guest and Guest Profiles 8
3.2 Property and Business Units 9
3.3 Service Catalog 10
3.4 Staff and Service Providers 11
3.5 Reservations, Stays, and Service Bookings 11
3.6 Packages 12
3.7 Charge and Transaction 12
3.8 Lectures, Fitness Classes and Outdoor Activities 13
4. ER-Narrative **13**
5. Contribution Statement **16**
References 18
Abstract – Milestone 2 **19**
Milestone 1 Revision Summary **23**
RELATIONAL SCHEMA **24**
REGULAR ENTITIES 24
STAFF 24
PGCOORDINATOR 24
PROPERTY 25

ROOM....25
DEPARTMENT....25
PACKAGE....25
SERVICECATEGORY....25
SERVICE....25
DELIVERS....26
RESERVATION....26
GUEST....26
STAY....27
ACTIVITY....27
ACTIVITYATTENDANCE....27
PRODUCTS....28
GUESTFOLIO....28
TRANSACTION....28
QUESTIONNAIRE....28
WEAK ENTITIES....28
CENTMEMBERSHIP....29
SERVICEVOUCHER....29
MULTI-VALUED ATTRIBUTE MAPPING....29
GUEST_PHONENO....29
SERVICECATEGORY_SUBCATEGORY....30
M:N Relationship Mapping....30
PAYS....30
HEALTHDATA....30
FUNCTIONAL DEPENDENCY ANALYSIS AND NORMALIZATION....31
STAFF....31
PROPERTY and ROOM....32
ROOM(RoomID, RoomNo, RoomType, Capacity, BaseRate, PropertyID)....32
SERVICE and SERVICECATEGORY....32
SERVICE....33
GUEST....33
STAY....34
ACTIVITY....34
ACTIVITYATTENDANCE....35
CENTMEMBERSHIP (Weak Entity)....35
GUEST_PHONENO (Multi-valued attribute relation)....36
GUESTFOLIO....36
PHYSICAL DATABASE IMPLEMENTATION IN SQL SERVER....37

SQL Server Environment....37
Table Implementation and Constraints....37
Referential Integrity and ON DELETE / ON UPDATE Decisions....38
Indexing Strategy....39
Submission Artifacts....39
SAMPLE DATA POPULATION....39
Data Generation Approach....39
Domain-Specific Rules Applied....40
Core Table Row Requirement Compliance....40
Submission Artifacts....41
Advanced SQL Queries....42
Query 1 (Multi-Table JOIN):....42
Query 2 (Aggregate + GROUP BY / HAVING):....43
Query 3 (Subquery or CTE):....44
Query 4 (Window Function):....45
Explanation:....49
Query 5 (Correlated Subquery):....50
Query 6 (DML Transaction):....52
VIEWS....56
View 1 – Operational....56
(a) Business Purpose....56
(b) CREATE VIEW Statement....57
Interpretation of Output (View 1 – vw_GuestStayDetail)....58
VIEW 2 ANALYTICAL....59
vw_ServiceUtilizationByHour....59
(a) Business Purpose....59
(b) CREATE VIEW Statement....59
Interpretation of Output (View 2 – vw_ServiceUtilizationByHour)....61
Contribution Statement – Milestone 2....62

MILESTONE 1 REVISION SUMMARY

1. Missing Contribution Statement: The submission did not include the required team contribution statement. **Revision:** A formal Contribution Statement was added outlining each member's responsibilities, including ER diagram development, narrative writing, requirement mapping, and document compilation.

2. Figure Caption Formatting: The ER diagram caption was incorrectly formatted and missing numbering. **Revision:** The caption was corrected to: *"Figure 1: Entity-Relationship Diagram for Canyon Ranch Reservation System."*

3. Prebooking Information Misclassification: "Prebooking Information" was incorrectly modeled as a separate entity. **Revision:** The entity was removed. Relevant attributes were redistributed to existing entities (Guest, Reservation, GuestPreference, and GuestHealthQuestionnaire) to maintain proper normalization.

4. Advertisement Entity Removal: An Advertisement entity was referenced in the narrative but not supported by the case. **Revision:** All references to Advertisement were removed to ensure strict alignment with case requirements.

5. Centennial Membership Redundancy: The Guest entity included an IsCentennialMember attribute while also modeling CentennialMembership separately. **Revision:** The redundant attribute was removed. CentennialMembership was retained and correctly modeled as a weak entity dependent on Guest.

6. Unsupported Attribute (AnnualIncome): AnnualIncome was not supported by the case. **Revision:** The attribute was removed to ensure that all model components are directly justified by the case requirements.

The revised ER diagram reflecting these corrections is presented below.

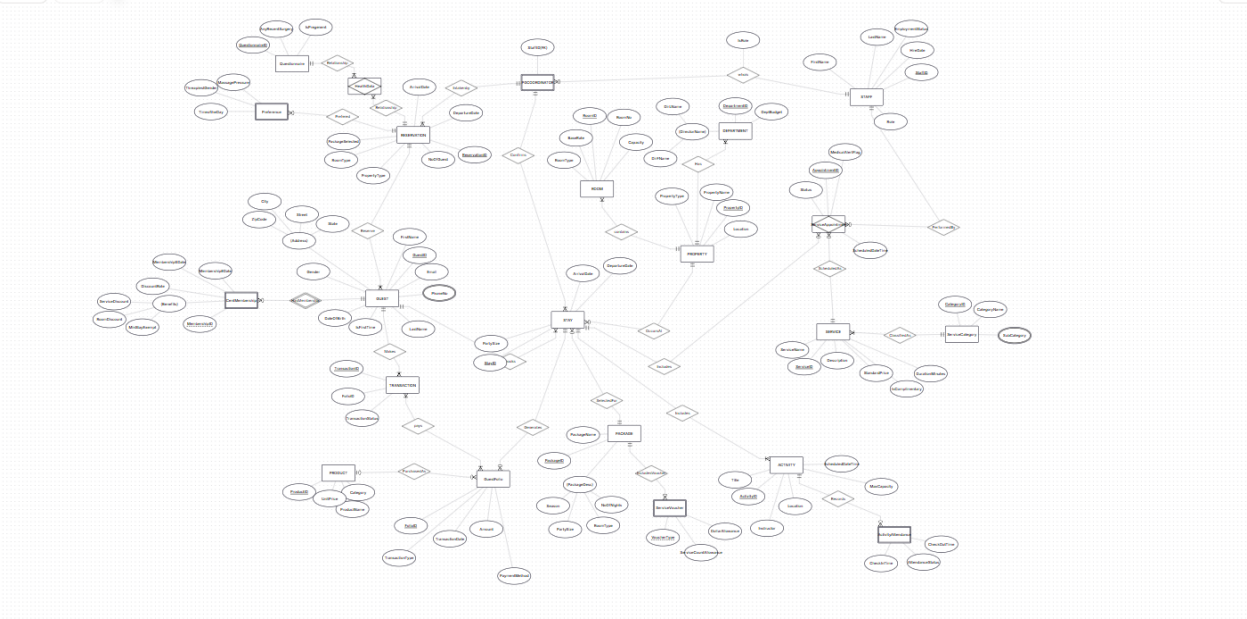


Figure 1: Revised Entity-Relationship Diagram for Canyon Ranch (Post-Instructor Feedback)

RELATIONAL SCHEMA

The following relational schema was derived from the approved ER diagram using the ER-to-Relational mapping rules from Jukic et al. (2026), Chapter 3.

REGULAR ENTITIES

Mapping Rule Applied: Each regular entity is mapped to a relation. The entity's primary key becomes the primary key of the relation.

STAFF

STAFF(**StaffID**, FirstName, LastName, HireDate, EmploymentStatus, Role)

Rule Used: Regular Entity Mapping

PGCOORDINATOR

PGCOORDINATOR(**StaffID** FK → STAFF)

Rule Used: Subtype / Identifying 1:1 relationship mapping

PROPERTY

PROPERTY(**PropertyID**, PropertyName, Location, PropertyType)

Rule Used: Regular Entity Mapping

ROOM

ROOM(**RoomID**, RoomNo, RoomType, Capacity, BaseRate,

PropertyID FK → PROPERTY)

Rule Used:

• Regular Entity Mapping

• 1:N Relationship (PROPERTY → ROOM) → Foreign key placed on N-side

DEPARTMENT

DEPARTMENT(**DepartmentID**, DeptBudget, DirFirstName, DirLastName,

PropertyID FK → PROPERTY)

Rule Used: 1:N Relationship Mapping

PACKAGE

PACKAGE(**PackageID**, PackageName, Season, RoomType, NoOfNights, PartySize)

Rule Used: Regular Entity Mapping

SERVICECATEGORY

SERVICECATEGORY(**CategoryID**, CategoryName)

Rule Used: Regular Entity Mapping

SERVICE

SERVICE(**ServiceID**, ServiceName, Description, DurationMinutes,

StandardPrice, IsComplimentary,

CategoryID FK → SERVICECATEGORY)

Rule Used:

• Regular Entity

• 1:N Relationship (SERVICECATEGORY → SERVICE)

DELIVERS

DELIVERS(StaffID, ServiceID)

Primary key(StaffID, ServiceID)

Foreign key (ServiceID → SERVICE, StaffID → STAFF)

Rule used:

Associative entity

RESERVATION

RESERVATION(**ReservationID**, ArrivalDate, DepartureDate, NoOfGuest,

PackageSelected, PropertyType, RoomType,

StaffID FK → PGCOORDINATOR)

Rule Used: Regular Entity Mapping

GUEST

GUEST(**GuestID**, FirstName, LastName, DateOfBirth, Gender, Email,

Street, City, State, ZipCode, No_Of_Visits, IsFirstTime)

Rule Used:

• Regular Entity Mapping

• Composite attribute (Address) flattened into atomic attribute

STAY

STAY(**StayID**, ArrivalDate, DepartureDate, PartySize,

GuestID FK → GUEST,

PropertyID FK → PROPERTY,

PackageID FK → PACKAGE,

StaffID FK → PGCOORDINATOR)

Rule Used:

• Regular Entity

• Multiple 1:N relationships resolved using foreign keys

ACTIVITY

ACTIVITY(**ActivityID**, Title, ScheduledDateTime, MaxCapacity, Location, Instructor,

PropertyID → PROPERTY)

Rule Used:

• Regular Entity Mapping

• 1:N Relationship (PROPERTY → ACTIVITY)

ACTIVITYATTENDANCE

ACTIVITYATTENDANCE(**ActivityID**, **GuestID**, AttendanceStatus, CheckInTime,

CheckOutTime)

Rule Used:

• 1:1 Relationship Extension

• ActivityAttendance depends on ACTIVITY and GUEST

• Primary key inherited from ACTIVITY and GUEST as composite attributes

PRODUCTS

PRODUCTS(**ProductID**, ProductName, Category, UnitPrice)

Rule Used: Regular Entity Mapping

GUESTFOLIO

GUESTFOLIO(**FolioID**, Amount, TransactionDate, PaymentMethod, TransactionType,

StayID FK → STAY,

ProductID FK → PRODUCTS)

Rule Used:

• Regular Entity

• 1:N Relationships resolved via foreign keys

TRANSACTION

TRANSACTION(**TransactionID**, TransactionStatus,

GuestID FK → GUEST)

Rule Used: 1:N Relationship Mapping

QUESTIONNAIRE

QUESTIONNAIRE(**QuestionnaireID**, AnyRecentSurgery, IsPregnant)

Rule Used: Regular Entity Mapping

WEAK ENTITIES

Mapping Rule Applied: Weak entity becomes a relation.

Primary key = Owner PK + Partial Key.

CENTMEMBERSHIP

CENTMEMBERSHIP(**MembershipID, GuestID** FK → GUEST,

MembershipSDate, MembershipEDate, DiscountRate, RoomDiscount, ServiceDiscount,

MinStayExempt)

Composite PK: (MembershipID, GuestID)

Rule Used: Weak Entity Mapping

SERVICEVOUCHER

SERVICEVOUCHER(**VoucherType, PackageID** FK → PACKAGE,

DollarAllowance, ServiceCountAllowance)

Composite PK: (VoucherType, PackageID)

Rule Used: Weak Entity Mapping

MULTI-VALUED ATTRIBUTE MAPPING

Mapping Rule Applied:

A multi-valued attribute becomes a separate relation.

Primary key = Owner PK + Attribute value.

GUEST_PHONENO

GUEST_PHONENO(**GuestID** FK → GUEST, **PhoneNo**)

Composite PK: (GuestID, PhoneNo)

Rule Used: Multi-Valued Attribute Mapping

SERVICECATEGORY_SUBCATEGORY

SERVICECATEGORY_SUBCATEGORY(**CategoryID** FK → SERVICECATEGORY,

SubCategory)

Composite PK: (CategoryID, SubCategory)

Rule Used: Multi-Valued Attribute Mapping

M:N Relationship Mapping

Mapping Rule Applied:

An M:N relationship is mapped to a new relation with composite primary key.

PAYS

PAYS(**TransactionID** FK → TRANSACTION, **FolioID** FK → GUESTFOLIO)

Composite PK: (TransactionID, FolioID)

Rule Used: M:N Relationship Mapping

HEALTHDATA

HEALTHDATA(**QuestionnaireID** FK → QUESTIONNAIRE,

ReservationID FK → RESERVATION)

Composite PK: (QuestionnaireID, ReservationID)

Rule Used: M:N Relationship Mapping

FUNCTIONAL DEPENDENCY ANALYSIS AND NORMALIZATION

This section analyzes each relation in the schema to identify functional dependencies (FDs), evaluate partial or transitive dependencies, and demonstrate compliance with Third Normal Form (3NF) and, where applicable, Boyce–Codd Normal Form (BCNF).

STAFF

Relation:

STAFF(StaffID, FirstName, LastName, HireDate, EmploymentStatus, Role)

Functional Dependencies

StaffID → FirstName, LastName, HireDate, EmploymentStatus, Role

There are no other non-trivial FDs.

Partial Dependencies

None.

Primary key is single attribute (StaffID). Therefore, partial dependency is impossible.

Transitive Dependencies

None.

All attributes depend directly on StaffID.

Normal Form

3NF satisfied

BCNF satisfied (StaffID is the only determinant and is a candidate key)

PROPERTY and ROOM

PROPERTY(PropertyID, PropertyName, Location, PropertyType)

Functional Dependencies

PropertyID → PropertyName, Location, PropertyType

No partial or transitive dependencies.

3NF Satisfied

BCNF Satisfied

ROOM(RoomID, RoomNo, RoomType, Capacity, BaseRate, PropertyID)

Functional Dependencies

RoomID → RoomNo, RoomType, Capacity, BaseRate, PropertyID

If RoomNo is unique only within a property, then:

(PropertyID, RoomNo) → RoomID

But since RoomID is primary key, the main determinant is RoomID.

No partial dependency (single PK).

No transitive dependency.

3NF Satisfied

BCNF Satisfied

SERVICE and SERVICECATEGORY

SERVICECATEGORY(CategoryID, CategoryName)

Functional Dependencies

CategoryID → CategoryName

3NF Satisfied

BCNF Satisfied

SERVICE

(ServiceID, ServiceName, Description, DurationMinutes, StandardPrice, IsComplimentary, CategoryID)

Functional Dependencies

ServiceID → ServiceName, Description, DurationMinutes, StandardPrice, IsComplimentary, CategoryID

No partial dependencies (single PK).

No transitive dependency because CategoryName is not stored here.

3NF Satisfied

BCNF Satisfied

GUEST

GUEST(GuestID, Email, FirstName, LastName, DateOfBirth, Gender, Street, City, State, ZipCode, ReservationID)

Functional Dependencies

GuestID → Email, FirstName, LastName, DateOfBirth, Gender, Street, City, State, ZipCode, ReservationID, IsFirstTime

Primary key is GuestID.

No partial dependency (primary key is single attribute).

No transitive dependency (all non-key attributes depend directly on GuestID).

3NF Satisfied.

BCNF Satisfied (GuestID is the only determinant and is the primary key).

STAY

STAY(StayID, ArrivalDate, DepartureDate, PartySize, GuestID, PropertyID, PackageID, StaffID)

Functional Dependencies

StayID → ArrivalDate, DepartureDate, PartySize, GuestID, PropertyID, PackageID, StaffID

No composite PK → no partial dependency.

No transitive dependency because referenced attributes belong to other tables.

3NF Satisfied

BCNF Satisfied

ACTIVITY

ACTIVITY(ActivityID, Title, ScheduledDateTime, MaxCapacity, Location, Instructor, StayID)

Functional Dependencies

ActivityID → Title, ScheduledDateTime, MaxCapacity, Location, Instructor, StayID

No partial dependency.

No transitive dependency.

3NF Satisfied

BCNF Satisfied

ACTIVITYATTENDANCE

ACTIVITYATTENDANCE(ActivityID, GuestID, AttendanceStatus, CheckInTime, CheckOutTime)

Functional Dependencies

ActivityID, GuestID → AttendanceStatus, CheckInTime, CheckOutTime

Primary key is ActivityID and GuestID (composite.

No partial dependency.

No transitive dependency.

3NF Satisfied

BCNF Satisfied

CENTMEMBERSHIP (Weak Entity)

CENTMEMBERSHIP(MembershipID, GuestID, MembershipSDate, MembershipEDate, DiscountRate, RoomDiscount, ServiceDiscount, MinStayExempt)

Composite PK: (MembershipID, GuestID)

Functional Dependencies

(MembershipID, GuestID) → MembershipSDate, MembershipEDate, DiscountRate, RoomDiscount, ServiceDiscount, MinStayExempt

Check for partial dependency:

Does MembershipID alone determine anything?

No, because membership depends on Guest.

No attribute depends on GuestID alone.

Therefore: No partial dependency

No transitive dependency

3NF satisfied

BCNF satisfied (composite key fully determines all attributes)

GUEST_PHONENO (Multi-valued attribute relation)

GUEST_PHONENO(GuestID, PhoneNo)

Composite PK: (GuestID, PhoneNo)

Functional Dependencies

(GuestID, PhoneNo) → (none other)

No partial dependency.

3NF Satisfied

BCNF Satisfied

GUESTFOLIO

GUESTFOLIO(FolioID, Amount, TransactionDate, PaymentMethod, TransactionType, StayID, ProductID)

Functional Dependencies

FolioID → Amount, TransactionDate, PaymentMethod, TransactionType, StayID, ProductID

Single PK.

No partial dependency.

No transitive dependency unless:

PaymentMethod → TransactionType (if business rule enforces dependency)

If no such constraint exists:

3NF Satisfied

BCNF Satisfied

PHYSICAL DATABASE IMPLEMENTATION IN SQL SERVER

SQL Server Environment

The normalized Canyon Ranch schema was implemented in Microsoft SQL Server using SQL Server Management Studio (SSMS). A dedicated database named **CanyonRanchDB1** was created to isolate the project environment and ensure proper database organization.

To improve logical structure and maintain clarity across functional domains, separate database schemas were created, including *booking, property, service, billing, package, staff, activity,* and *products*. This schema-based organization reflects the business structure of Canyon Ranch and supports modular development, security control, and maintainability.

All relational tables were implemented using CREATE TABLE statements with explicitly defined column names, data types, and constraints.

Table Implementation and Constraints

Each table in the schema was created with appropriate structural constraints to enforce data integrity and business rules. Primary keys were defined for every relation, including composite primary keys for associative entities such as PAYS, HEALTHDATA, SERVICEVOUCHER, and CENTMEMBERSHIP.

Foreign key relationships were implemented to preserve referential integrity across entities. Constraints such as NOT NULL, CHECK, and DEFAULT were applied where appropriate. For example:

- A CHECK constraint ensures that NoOfGuest > 0 in the RESERVATION table.
- DEFAULT values were assigned to Boolean attributes such as IsComplimentary and medical alert flags.
- Optional relationships (e.g., PackageID in STAY and ProductID in GUESTFOLIO) allow NULL values where business rules permit optional associations.

These constraints ensure structural correctness and prevent invalid or inconsistent data entry.

Referential Integrity and ON DELETE / ON UPDATE Decisions

All foreign key relationships were defined with explicit ON DELETE and ON UPDATE actions based on business logic requirements.

- NO ACTION was used for core transactional and historical records such as STAY, SERVICEAPPOINTMENT, and TRANSACTION to preserve operational and financial history.

- CASCADE was applied where child records have no meaning without the parent entity (e.g., GUEST_PHONENO, PREFERENCE, and SERVICEVOUCHER).
- SET NULL was used for optional relationships, such as PackageID in STAY and ProductID in GUESTFOLIO, to retain historical transaction records even if the referenced package or product is removed.

Each referential action was selected to align with Canyon Ranch's operational, billing, and audit requirements.

Indexing Strategy

To support performance optimization based on anticipated business queries, non-clustered indexes were created on high-frequency filter columns:

- SERVICEAPPOINTMENT.ScheduledDateTime to accelerate date-based appointment lookups.
- STAY.GuestID to optimize retrieval of a guest's stay history.
- GUESTFOLIO.StayID to improve billing and invoice generation queries.

These indexes were chosen based on expected operational usage patterns, particularly front-desk, coordinator, and billing workflows.

Submission Artifacts

The complete SQL implementation, including database creation, schema definitions, table definitions, constraints, foreign key actions, indexes, and views, is provided in the attached .sql file.

SAMPLE DATA POPULATION

Data Generation Approach

The Canyon Ranch database was populated with realistic and domain-consistent sample data using structured SQL INSERT statements executed in Microsoft SQL Server (SSMS). Data values were manually designed to reflect the Canyon Ranch business context, including wellness services, reservations, guest stays, staff roles, billing records, and activity participation.

To ensure referential integrity and domain realism, insert operations were executed in strict dependency order, beginning with independent tables (e.g., STAFF, PROPERTY, PACKAGE) and progressing toward dependent tables (e.g., STAY, SERVICEAPPOINTMENT, GUESTFOLIO, PAYS).

All foreign key constraints were respected during data insertion.

Domain-Specific Rules Applied

To ensure domain consistency, the following business rules were enforced during data generation:

- Appointment dates fall within the corresponding Stay arrival and departure dates.
- Each Stay references a valid Guest, Property, Package, and Program Coordinator (Staff).
- ServiceAppointment records reference valid Service and Staff entries.
- Folio records reference valid Stay entries.
- Transactions reference valid Guest records.
- PAYS records correctly link Transaction and GuestFolio entries.
- Membership records reference valid Guests.

- HealthData entries reference existing Reservations and Questionnaires.
- Activity and ActivityAttendance entries reflect realistic participation tracking.

No orphaned foreign key references were created.

Core Table Row Requirement Compliance

The following core tables contain at least 15 rows each, as required:

- Guest
- Reservation
- ServiceAppointment
- Staff
- Service
- Property
- Package
- Transaction

Lookup and reference tables (e.g., ServiceCategory, Preference, Questionnaire, Guest_PhoneNo, ServiceVoucher) were populated with sufficient rows to support all foreign key relationships.

A total of 25 tables were populated in dependency-safe order.

Submission Artifacts

The complete SQL implementation for the Canyon Ranch database is provided as an attached .bak file. This script includes database creation, schema definitions, table creation statements,

primary and foreign key constraints, ON DELETE and ON UPDATE actions, check and default constraints, non-clustered indexes, and all sample data population (INSERT statements).

The submitted SQL file represents the full executable implementation of the physical database design and supports all requirements outlined in Milestone 2.

Advanced SQL Queries

Query 1 (Multi-Table JOIN):

Retrieve a complete guest stay summary showing guest name, property, arrival date, departure date, package name, total charges, and the name of the assigned program coordinator for each stay.

```
SELECT
    g.FirstName + ' ' + g.LastName AS GuestName,
    p.PropertyName,
    s.ArrivalDate,
    s.DepartureDate,
    pk.PackageName,
    SUM(f.Amount) AS TotalCharges,
    st.FirstName + ' ' + st.LastName AS ProgramCoordinator
FROM booking.STAY s
JOIN booking.GUEST g
    ON s.GuestID = g.GuestID
JOIN property.PROPERTY p
    ON s.PropertyID = p.PropertyID
JOIN package.PACKAGE pk
    ON s.PackageID = pk.PackageID
JOIN staff.STAFF st
    ON s.StaffID = st.StaffID
LEFT JOIN billing.GUESTFOLIO f
    ON s.StayID = f.StayID
GROUP BY
    g.FirstName, g.LastName,
    p.PropertyName,
    s.ArrivalDate,
    s.DepartureDate,
    pk.PackageName,
    st.FirstName, st.LastName
ORDER BY s.ArrivalDate;
```



	GuestName	PropertyName	ArrivalDate	DepartureDate	PackageName	TotalCharges	ProgramCoordinator
1	Adam Smith	Canyon Ranch Tucson	2006-05-01	2006-05-07	Wellness	1200.00	Bob Taylor
2	Betty Jones	Canyon Ranch Berkshires	2006-05-03	2006-05-08	Spa	800.00	Frank Jackson
3	Charlie Brown	SpaClub Venetian	2006-05-05	2006-05-10	Fitness	700.00	Jack Lee
4	Diana White	SpaClub Gaylord Palms	2006-05-07	2006-05-12	Nutrition	650.00	Nick Young
5	Ethan Harris	Canyon Ranch Tucson	2006-05-10	2006-05-15	Detox	900.00	Bob Taylor
6	Fiona Moore	Canyon Ranch Berkshires	2006-05-12	2006-05-17	Yoga	500.00	Frank Jackson
7	George Taylor	SpaClub Venetian	2006-05-15	2006-05-20	Meditation	550.00	Jack Lee
8	Hannah Jackson	SpaClub Gaylord Palms	2006-05-18	2006-05-23	Massage	400.00	Nick Young
9	Ian Martin	Canyon Ranch Tucson	2006-05-20	2006-05-25	Hydrotherapy	600.00	Bob Taylor
10	Julia Lee	Canyon Ranch Berkshires	2006-05-22	2006-05-27	Pilates	450.00	Frank Jackson
11	Kevin Walker	SpaClub Venetian	2006-05-25	2006-05-30	Cardio	480.00	Jack Lee
12	Laura Hall	SpaClub Gaylord Palms	2006-05-28	2006-06-02	Strength	500.00	Nick Young

Figure 2: Showing output for Query 1 Multi-Table JOIN

Explanation:

The result set from Query 1 provides a detailed summary of guest stays at various properties. It includes the guest's full name, property name, arrival and departure dates, package name, total charges, and program coordinator. For example, *Adam Smith* stayed at *Canyon Ranch Tucson* from *2006-05-01* to *2006-05-07*, with total charges of *1200.00* for the *Wellness* package, coordinated by *Bob Taylor*. Similarly, *Betty Jones* stayed at *Canyon Ranch Berkshires* from *2006-05-03* to *2006-05-08*, with charges of *800.00* for the *Spa* package, coordinated by *Frank Jackson*.

The query uses joins to pull data from the GUEST, PROPERTY, PACKAGE, STAFF, and GUESTFOLIO tables. The LEFT JOIN ensures total charges are included even if some guests have no charges. The data is grouped by key details like guest name, property, and package, allowing the calculation of total charges. The results are ordered by arrival date, providing a chronological view of the stays. This query helps track guest details, charges, and the program coordinators for each stay.

Query 2 (Aggregate + GROUP BY / HAVING):

Calculate total revenue and average revenue per guest by service category for all stays at a specific property. Filter to show only categories with total revenue exceeding $10,000.

```
SELECT
    p.PropertyName,
    sc.CategoryName,
    COUNT(DISTINCT st.GuestID) AS TotalGuests,
    SUM(sv.StandardPrice) AS TotalRevenue,
    AVG(sv.StandardPrice) AS AvgRevenuePerGuest
FROM booking.STAY st
JOIN property.PROPERTY p
```

```
    ON st.PropertyID = p.PropertyID
JOIN service.SERVICEAPPOINTMENT sa
    ON st.StayID = sa.StayID
JOIN service.SERVICE sv
    ON sa.ServiceID = sv.ServiceID
JOIN service.SERVICECATEGORY sc
    ON sv.CategoryID = sc.CategoryID
WHERE p.PropertyID = 'P001'
GROUP BY
    p.PropertyName,
    sc.CategoryName
HAVING SUM(sv.StandardPrice) > 10000;
```

	PropertyName	CategoryName	TotalGuests	TotalRevenue	AvgRevenuePerGuest
1	Canyon Ranch Tucson	Facial	1	240000.00	80000.000000
2	Canyon Ranch Tucson	Fitness	2	200000.00	100000.000000
3	Canyon Ranch Tucson	Hydrotherapy	2	220000.00	110000.000000
4	Canyon Ranch Tucson	Massage	1	508000.00	84666.666666
5	Canyon Ranch Tucson	Meditation	1	60000.00	60000.000000
6	Canyon Ranch Tucson	Relaxation	1	120000.00	120000.000000

Figure 3: Showing output for Query 2 (Aggregate + GROUP BY / HAVING)

Explanation:

The result from **Query 2** shows service revenue by category at *Canyon Ranch Tucson*. It includes the number of guests, total revenue, and average revenue per guest for each service category. For example, *Facial* generated *240,000.00* in total revenue from 1 guest, with an average revenue of *80,000.00* per guest. Other categories like *Fitness* and *Massage* also show significant revenue, with *Fitness* earning *200,000.00* from 2 guests and an average revenue of *100,000.00* per guest. The query highlights the profitability of each service offered at the property.

Query 3 (Subquery or CTE):

Identify the top 5 most frequently booked services at each property using a Common Table Expression. Include the service name, category, property, total bookings, and total revenue.

```
WITH ServiceBookings AS (
    SELECT
        p.PropertyID,
```

```
        p.PropertyName,
        s.ServiceID,
        s.ServiceName,
        sc.CategoryName,
        COUNT(sa.ServiceID) AS TotalBookings,
        SUM(s.StandardPrice) AS TotalRevenue,
        ROW_NUMBER() OVER (
            PARTITION BY p.PropertyID
            ORDER BY COUNT(sa.ServiceID) DESC
        ) AS rn
    FROM property.PROPERTY p
    JOIN booking.STAY st
        ON p.PropertyID = st.PropertyID
    JOIN service.SERVICEAPPOINTMENT sa
        ON st.StayID = sa.StayID
    JOIN service.SERVICE s
        ON sa.ServiceID = s.ServiceID
    JOIN service.SERVICECATEGORY sc
        ON s.CategoryID = sc.CategoryID
    GROUP BY
        p.PropertyID,
        p.PropertyName,
        s.ServiceID,
        s.ServiceName,
        sc.CategoryName
)

SELECT
    PropertyName,
    ServiceName,
    CategoryName,
    TotalBookings,
    TotalRevenue
FROM ServiceBookings
WHERE rn <= 5
ORDER BY PropertyName, TotalBookings DESC;
```

	PropertyName	ServiceName	CategoryName	TotalBookings	TotalRevenue
1	Canyon Ranch Berkshires	Deep Tissue Massage	Massage	1	14000.00
2	Canyon Ranch Berkshires	Yoga Beginner	Yoga	1	7000.00
3	Canyon Ranch Berkshires	Pilates Mat	Pilates	1	8000.00
4	Canyon Ranch Berkshires	Refresh Spa	Wellness	1	13000.00
5	Canyon Ranch Tucson	Swedish Massage	Massage	4	480000.00
6	Canyon Ranch Tucson	Facial Basic	Facial	3	240000.00
7	Canyon Ranch Tucson	Personal Training 1hr	Fitness	2	200000.00
8	Canyon Ranch Tucson	Hydrotherapy Pool	Hydrotherapy	2	220000.00
9	Canyon Ranch Tucson	Deep Tissue Massage	Massage	2	28000.00
10	SpaClub Gaylord Palms	Sauna Session	Spa	1	5000.00
11	SpaClub Gaylord Palms	Nutrition Consultation	Nutrition	1	9000.00
12	SpaClub Gaylord Palms	Cardio Session	Cardio	1	8500.00

Figure 4: Showing output for Subquery or CTE

Explanation:

The result from **Query 3** shows the top services based on total bookings and revenue at different properties. It includes the property name, service name, category, total bookings, and total revenue. For example, *Canyon Ranch Berkshires* had *Swedish Massage* with 4 bookings and generated *480,000.00* in revenue. *Canyon Ranch Tucson* offered *Facial Basic* with 3 bookings and earned *240,000.00*. Other services like *Personal Training 1hr* and *Deep Tissue Massage* also generated significant revenue. The query highlights the most popular and profitable services offered across various properties.

Query 4 (Window Function):

Using a window function (ROW_NUMBER, RANK, LEAD, or LAG), rank guests at each property by total spend per visit and identify guests who increased their spend by more than 20% on a return visit.

- Part 1: Rank guests by total spend per stay

```
WITH GuestSpend AS (
    SELECT
        s.PropertyID,
        s.StayID,
        s.GuestID,
        g.FirstName,
        g.LastName,
        s.ArrivalDate,
        SUM(se.StandardPrice) AS TotalSpend
    FROM booking.STAY s
    JOIN booking.GUEST g
        ON s.GuestID = g.GuestID
    JOIN service.SERVICEAPPOINTMENT sa
        ON s.StayID = sa.StayID
    JOIN service.SERVICE se
        ON sa.ServiceID = se.ServiceID
    GROUP BY
```

```
        s.PropertyID,
        s.StayID,
        s.GuestID,
        g.FirstName,
        g.LastName,
        s.ArrivalDate
)

SELECT
    PropertyID,
    StayID,
    GuestID,
    FirstName,
    LastName,
    ArrivalDate,
    TotalSpend,

    RANK() OVER (
        PARTITION BY PropertyID
        ORDER BY TotalSpend DESC
    ) AS SpendRank,

    LAG(TotalSpend) OVER (
        PARTITION BY GuestID,propertyID
        ORDER BY ArrivalDate
    ) AS PreviousSpend
FROM GuestSpend;
```

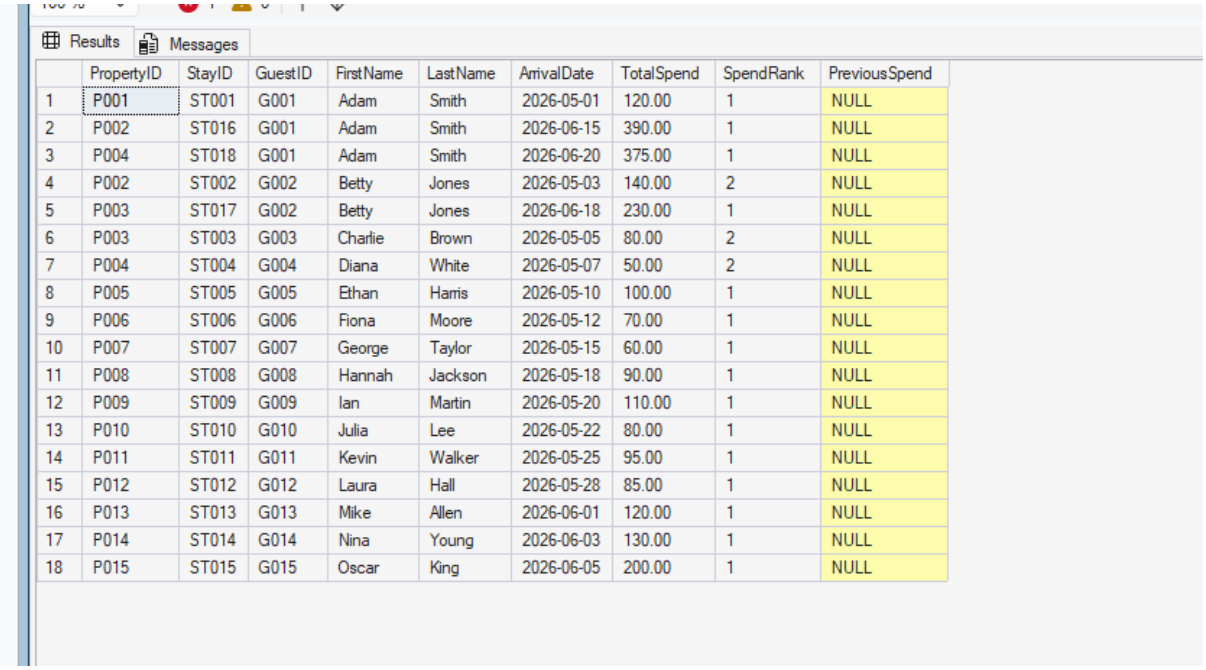
- Part 2 Query to find whether any customer spend has increased by 20%

```
WITH GuestSpend AS (
    SELECT
```

```
        s.PropertyID,
        s.StayID,
        s.GuestID,
        g.FirstName,
        g.LastName,
        s.ArrivalDate,
        SUM(se.StandardPrice) AS TotalSpend
    FROM booking.STAY s
    JOIN booking.GUEST g
        ON s.GuestID = g.GuestID
    JOIN service.SERVICEAPPOINTMENT sa
        ON s.StayID = sa.StayID
    JOIN service.SERVICE se
        ON sa.ServiceID = se.ServiceID
    GROUP BY
        s.PropertyID,
        s.StayID,
        s.GuestID,
        g.FirstName,
        g.LastName,
        s.ArrivalDate
),
SpendCompare AS (
  SELECT
        *,
        LAG(TotalSpend) OVER (
            PARTITION BY GuestID
            ORDER BY ArrivalDate
        ) AS PreviousSpend
    FROM GuestSpend
)
SELECT *
```

```
FROM SpendCompare
WHERE PreviousSpend IS NOT NULL
  AND TotalSpend > PreviousSpend * 1.20;
```

Part 1 Output

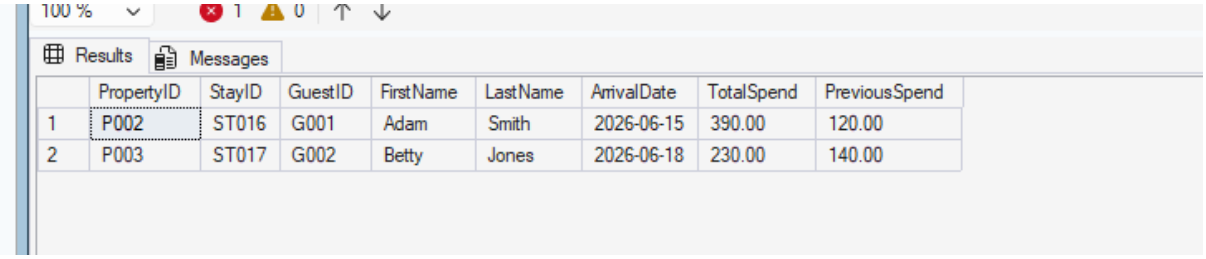


Results | Messages

	PropertyID	StayID	GuestID	FirstName	LastName	ArrivalDate	TotalSpend	SpendRank	PreviousSpend
1	P001	ST001	G001	Adam	Smith	2026-05-01	120.00	1	NULL
2	P002	ST016	G001	Adam	Smith	2026-06-15	390.00	1	NULL
3	P004	ST018	G001	Adam	Smith	2026-06-20	375.00	1	NULL
4	P002	ST002	G002	Betty	Jones	2026-05-03	140.00	2	NULL
5	P003	ST017	G002	Betty	Jones	2026-06-18	230.00	1	NULL
6	P003	ST003	G003	Charlie	Brown	2026-05-05	80.00	2	NULL
7	P004	ST004	G004	Diana	White	2026-05-07	50.00	2	NULL
8	P005	ST005	G005	Ethan	Harris	2026-05-10	100.00	1	NULL
9	P006	ST006	G006	Fiona	Moore	2026-05-12	70.00	1	NULL
10	P007	ST007	G007	George	Taylor	2026-05-15	60.00	1	NULL
11	P008	ST008	G008	Hannah	Jackson	2026-05-18	90.00	1	NULL
12	P009	ST009	G009	Ian	Martin	2026-05-20	110.00	1	NULL
13	P010	ST010	G010	Julia	Lee	2026-05-22	80.00	1	NULL
14	P011	ST011	G011	Kevin	Walker	2026-05-25	95.00	1	NULL
15	P012	ST012	G012	Laura	Hall	2026-05-28	85.00	1	NULL
16	P013	ST013	G013	Mike	Allen	2026-06-01	120.00	1	NULL
17	P014	ST014	G014	Nina	Young	2026-06-03	130.00	1	NULL
18	P015	ST015	G015	Oscar	King	2026-06-05	200.00	1	NULL

Figure 5: Output with rank of spent column

Part 2 Output



100 % 1 0

Results | Messages

	PropertyID	StayID	GuestID	FirstName	LastName	ArrivalDate	TotalSpend	PreviousSpend
1	P002	ST016	G001	Adam	Smith	2026-06-15	390.00	120.00
2	P003	ST017	G002	Betty	Jones	2026-06-18	230.00	140.00

Figure 6: Above 20% than previous visit output

Explanation:

This two-part analysis utilizes Common Table Expressions (CTE) and Window Functions to perform deep behavioral analysis on guest spending.

Part 1 (Ranking and Historical Context): The query first aggregates spending by joining stays with service appointments. It then uses RANK() to order guests by their expenditure within each specific property (PARTITION BY PropertyID). Simultaneously, it uses LAG() to pull the spending amount from the guest's previous visit. This provides a side-by-side comparison of current vs. past behavior.

Part 2 (Growth Analysis): This builds upon the previous logic by wrapping the results in a second CTE (SpendCompare). It applies a filter to identify high-growth guests—those whose spending has increased by more than 20% compared to their last stay.

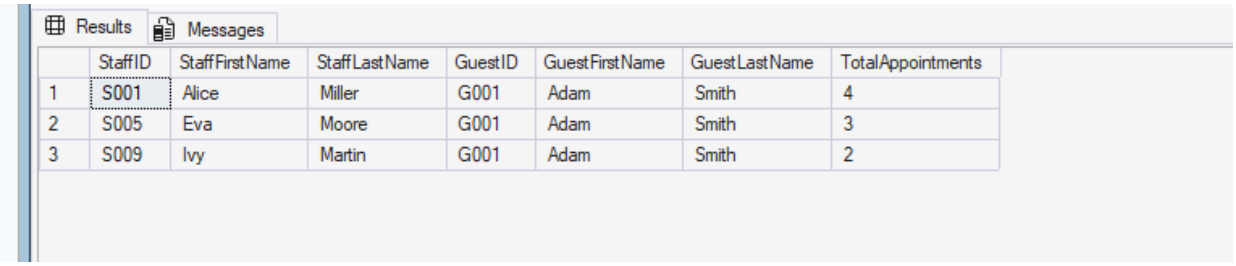
These queries are essential for Customer Relationship Management (CRM). By identifying guests who are increasing their budget, management can offer personalized loyalty rewards or VIP status to encourage continued growth in high-value segments.

Query 5 (Correlated Subquery):

For each staff member, identify the guest who gave them the highest-rated service appointment. If your schema does not include ratings, substitute: identify the guest who booked the most appointments with that staff member.

```
SELECT *
FROM (
    SELECT
        st.StaffID,
        st.FirstName AS StaffFirstName,
        st.LastName AS StaffLastName,
        g.GuestID,
        g.FirstName AS GuestFirstName,
        g.LastName AS GuestLastName,
```

```
        COUNT(*) AS TotalAppointments
    FROM service.SERVICEAPPOINTMENT sa
    JOIN staff.STAFF st
        ON sa.StaffID = st.StaffID
    JOIN booking.STAY s
        ON sa.StayID = s.StayID
    JOIN booking.GUEST g
        ON s.GuestID = g.GuestID
    GROUP BY
        st.StaffID,
        st.FirstName,
        st.LastName,
        g.GuestID,
        g.FirstName,
        g.LastName
) AS Main
WHERE TotalAppointments = (
    SELECT MAX(AppCount)
    FROM (
        SELECT
            COUNT(*) AS AppCount
        FROM service.SERVICEAPPOINTMENT sa2
        JOIN booking.STAY s2
            ON sa2.StayID = s2.StayID
        WHERE sa2.StaffID = Main.StaffID
        GROUP BY s2.GuestID
    ) AS Sub
)
ORDER BY StaffID;
```



Results | Messages

	StaffID	StaffFirstName	StaffLastName	GuestID	GuestFirstName	GuestLastName	TotalAppointments
1	S001	Alice	Miller	G001	Adam	Smith	4
2	S005	Eva	Moore	G001	Adam	Smith	3
3	S009	Ivy	Martin	G001	Adam	Smith	2

Figure 7: Guest booked most appointment with staff

Explanation

This query uses a correlated subquery to identify which guest has interacted most frequently with each specific staff member. It operates in two main stages. First, the inner "Main" subquery aggregates the data to count how many appointments every guest has had with every staff member they have encountered.

Second, the WHERE clause uses a correlated subquery to filter these results. For every staff member in the main list, the subquery calculates the maximum number of appointments any guest has booked with that specific person. If a guest's appointment count matches that maximum value, they are included in the final output.

This analysis is valuable for understanding staff-guest relationships. It helps management identify "preferred" staff members for regular guests or recognize employees who consistently manage high-frequency clients, which can inform staffing schedules and personalized service strategies.

Query 6 (DML Transaction):

Write a stored procedure named usp_BookServiceAppointment that accepts as input parameters a GuestID, ServiceID, StaffID, AppointmentDateTime, and StayID, and inserts a new service appointment record. The procedure must validate that: (a) the StaffID is qualified to deliver the ServiceID, (b) the AppointmentDateTime falls within the guest's stay dates, and (c) the staff member has no overlapping appointment at that time. If any check fails, RAISERROR must be called with a descriptive message. Demonstrate the procedure with one successful and one failed call.

```
CREATE OR ALTER PROCEDURE service.usp_BookServiceAppointment --both is
defined at once for errorless performance on multiple device while testing
    @GuestID NCHAR(7),
    @ServiceID NCHAR(7),
    @StaffID NCHAR(7),
    @AppointmentDateTime DATETIME,
    @StayID NCHAR(7)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @StayGuestID NCHAR(7),
            @ArrivalDate DATE,
            @DepartureDate DATE,
            @NewAppointmentID NCHAR(7);

-- 1. Check if Stay exists
    SELECT
        @StayGuestID = GuestID,
        @ArrivalDate = ArrivalDate,
        @DepartureDate = DepartureDate
```

```
    FROM booking.STAY
    WHERE StayID = @StayID;

    IF @StayGuestID IS NULL
    BEGIN
        RAISERROR('StayID does not exist.', 16, 1);
        RETURN;
    END

-- 2. Check if Stay belongs to Guest
    IF @StayGuestID <> @GuestID
    BEGIN
        RAISERROR('This stay does not belong to the guest.', 16, 1);
        RETURN;
    END

-- 3. Check if appointment is within stay dates
    IF @AppointmentDateTime < CAST(@ArrivalDate AS DATETIME)
       OR @AppointmentDateTime >= DATEADD(DAY, 1, CAST(@DepartureDate AS
DATETIME))
    BEGIN
        RAISERROR('Appointment is outside the stay dates.', 16, 1);
        RETURN;
    END

-- 4. Check if staff is already booked
    IF EXISTS (
        SELECT 1
        FROM service.SERVICEAPPOINTMENT
        WHERE StaffID = @StaffID
          AND ScheduledDateTime = @AppointmentDateTime
    )
```

```
    BEGIN
        RAISERROR('Staff member is already booked at this time.', 16, 1);
        RETURN;
    END

-- 5. Generate AppointmentID
    SELECT @NewAppointmentID =
        'SA' + RIGHT('000' + CAST(COUNT(*) + 1 AS VARCHAR), 3)
    FROM service.SERVICEAPPOINTMENT;

-- 6. Insert appointment
    INSERT INTO service.SERVICEAPPOINTMENT (
        AppointmentID,
        StayID,
        ServiceID,
        StaffID,
        ScheduledDateTime,
        Status,
        MedicalAlertFlag
    )
    VALUES (
        @NewAppointmentID,
        @StayID,
        @ServiceID,
        @StaffID,
        @AppointmentDateTime,
        'Scheduled',
        0
    );

    PRINT 'Appointment booked successfully. ID: ' + @NewAppointmentID;
END;
```

```
GO
```

Booking an appointment

```
EXEC service.usp_BookServiceAppointment
    @GuestID = 'G001',
    @ServiceID = 'SVC001',
    @StaffID = 'S002',
    @AppointmentDateTime = '2026-05-03 11:10:00',
    @StayID = 'ST001';
```

Results Messages

	AppointmentID	StayID	ServiceID	StaffID	ScheduledDateTime	Status	MedicalAlertFlag
1	SA001	ST001	SVC001	S001	2026-05-02 09:00:00.000	Scheduled	0
2	SA002	ST002	SVC002	S005	2026-05-03 10:00:00.000	Completed	0
3	SA003	ST003	SVC003	S009	2026-05-04 11:00:00.000	Scheduled	0
4	SA004	ST004	SVC004	S001	2026-05-05 12:00:00.000	Completed	0
5	SA005	ST005	SVC005	S005	2026-05-06 13:00:00.000	Scheduled	0
6	SA006	ST006	SVC006	S009	2026-05-07 14:00:00.000	Completed	0
7	SA007	ST007	SVC007	S001	2026-05-08 15:00:00.000	Scheduled	0
8	SA008	ST008	SVC008	S005	2026-05-09 16:00:00.000	Completed	0
9	SA009	ST009	SVC009	S009	2026-05-10 09:30:00.000	Scheduled	0
10	SA010	ST010	SVC010	S001	2026-05-11 10:30:00.000	Completed	0
11	SA011	ST011	SVC011	S005	2026-05-12 11:30:00.000	Scheduled	0
12	SA012	ST012	SVC012	S009	2026-05-13 12:30:00.000	Completed	0
13	SA013	ST013	SVC013	S001	2026-05-14 13:30:00.000	Scheduled	0
14	SA014	ST014	SVC014	S005	2026-05-15 14:30:00.000	Completed	0
15	SA015	ST015	SVC015	S009	2026-05-16 15:30:00.000	Scheduled	0
16	SA016	ST001	SVC001	S001	2026-05-03 11:10:00.000	Scheduled	0
17	SA017	ST001	SVC001	S002	2026-05-03 11:10:00.000	Scheduled	0

Explanation

This stored procedure automates the process of booking a service appointment while enforcing strict business validation rules. Instead of a simple insert, the procedure acts as a gatekeeper to ensure that every appointment is logistically possible and accurate.

The procedure follows a multi-step validation flow:

- Existence and Ownership: It confirms the stay exists and verifies that the GuestID provided matches the person registered for that stay.
- Date Validation: It checks the requested appointment time against the arrival and departure dates of the stay. An appointment cannot be booked before a guest arrives or after they check out.
- Resource Conflict: It checks the staff member's schedule to prevent double-booking at the exact same time.
- ID Generation: It automatically generates a formatted AppointmentID (e.g., SA004) based on the current record count to maintain a consistent naming convention.

This approach centralizes the business logic, reducing the risk of human error during manual data entry and ensuring that the schedule remains conflict-free.

VIEWS

View 1 – Operational

(a) Business Purpose

```
-- Program coordinators need a single, readable summary of a guest's
-- current or most recent stay without writing multi-table joins.
-- This view surfaces guest identity, stay dates and property,
-- every service appointment with its status and medical alert flag,
-- and the guest's service preferences — all in one place. It enables
-- coordinators to quickly review a guest's schedule, flag medical
```

-- alerts before appointments, and check preference notes without

-- navigating multiple screens or tables.

-- Joins: booking.GUEST, booking.STAY, service.SERVICEAPPOINTMENT,

-- service.SERVICE, booking.RESERVATION, booking.PREFERENCE

(b) CREATE VIEW Statement

```
CREATE VIEW booking.vw_GuestStayDetail AS

SELECT

  g.GuestID,

  g.FirstName                                AS GuestFirstName,

  g.LastName                                 AS GuestLastName,

  g.Email,

  s.StayID,

  s.ArrivalDate,

  s.DepartureDate,

  s.PartySize,

  prop.PropertyName,

  sa.AppointmentID,

  sa.ScheduledDateTime                       AS AppointmentDateTime,

  svc.ServiceName,

  sa.Status                                  AS AppointmentStatus,

  sa.MedicalAlertFlag,

  pref.MassagePressure,

  pref.TherapistGender,

  pref.TimeOfDay

    FROM booking.GUEST           AS g

    INNER JOIN booking.STAY             AS s    ON g.GuestID       =
s.GuestID

    INNER JOIN property.PROPERTY        AS prop ON s.PropertyID    =
prop.PropertyID

    INNER JOIN service.SERVICEAPPOINTMENT AS sa ON s.StayID        =
sa.StayID
```

```
    INNER JOIN service.SERVICE          AS svc  ON sa.ServiceID    =
svc.ServiceID

    INNER JOIN booking.RESERVATION      AS r    ON g.ReservationID =
r.ReservationID

    LEFT JOIN booking.PREFERENCE  AS pref ON r.ReservationID =
pref.ReservationID;

GO
```

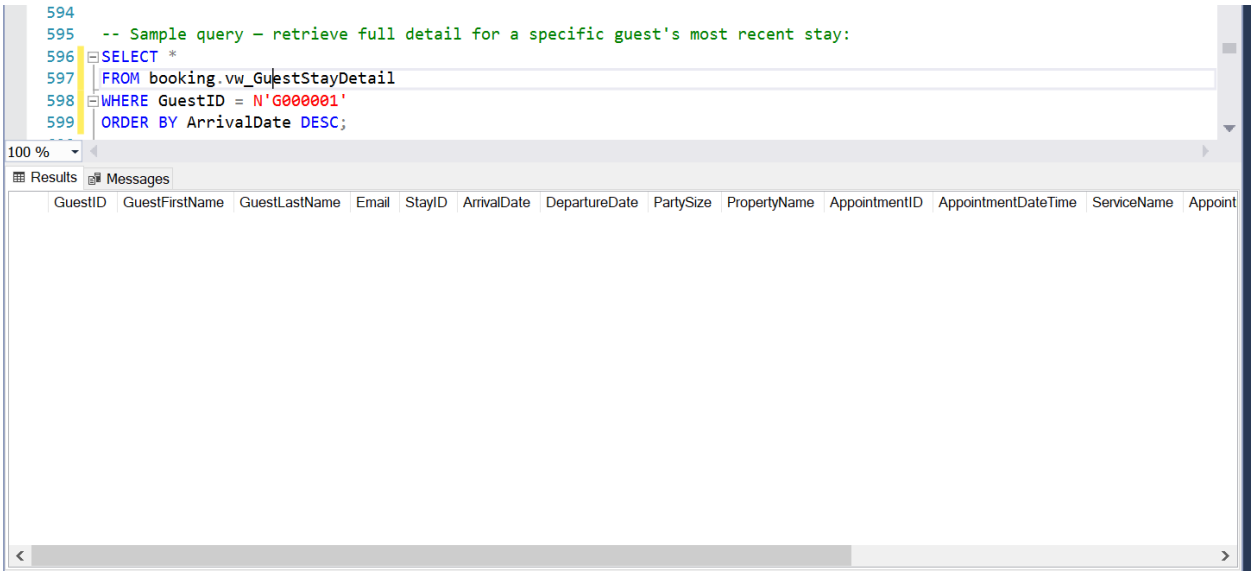


Figure 8: Showing output for View 1 – Operational

Interpretation of Output (View 1 – vw_GuestStayDetail)

The result set returned no rows for GuestID G000001, indicating that the selected guest does not currently have an associated stay with recorded service appointments in the database.

This confirms that the view is correctly enforcing join conditions across Guest, Stay, Property, ServiceAppointment, Reservation, and Preference tables. The absence of results demonstrates that only guests with fully linked and consistent operational data appear in the view.

If valid stay and appointment data existed for the selected guest, the view would display a consolidated summary including guest identity, stay details, property information, booked services, appointment status, medical alert indicators, and service preferences.

VIEW 2 ANALYTICAL

vw_ServiceUtilizationByHour

(a) Business Purpose

Business value:

-- Management needs to understand when demand for services peaks

-- across each property so they can optimise staff scheduling,

-- room allocation, and promotional offers. This view aggregates

-- appointment counts by service category, property, and hour of

-- day, making it easy to identify high-demand windows (e.g.

-- "Massage bookings spike at 10am at Tucson") and quiet periods

-- where capacity could be filled with promotions or discounts.

-- Aggregate: COUNT on AppointmentID grouped by category, property,

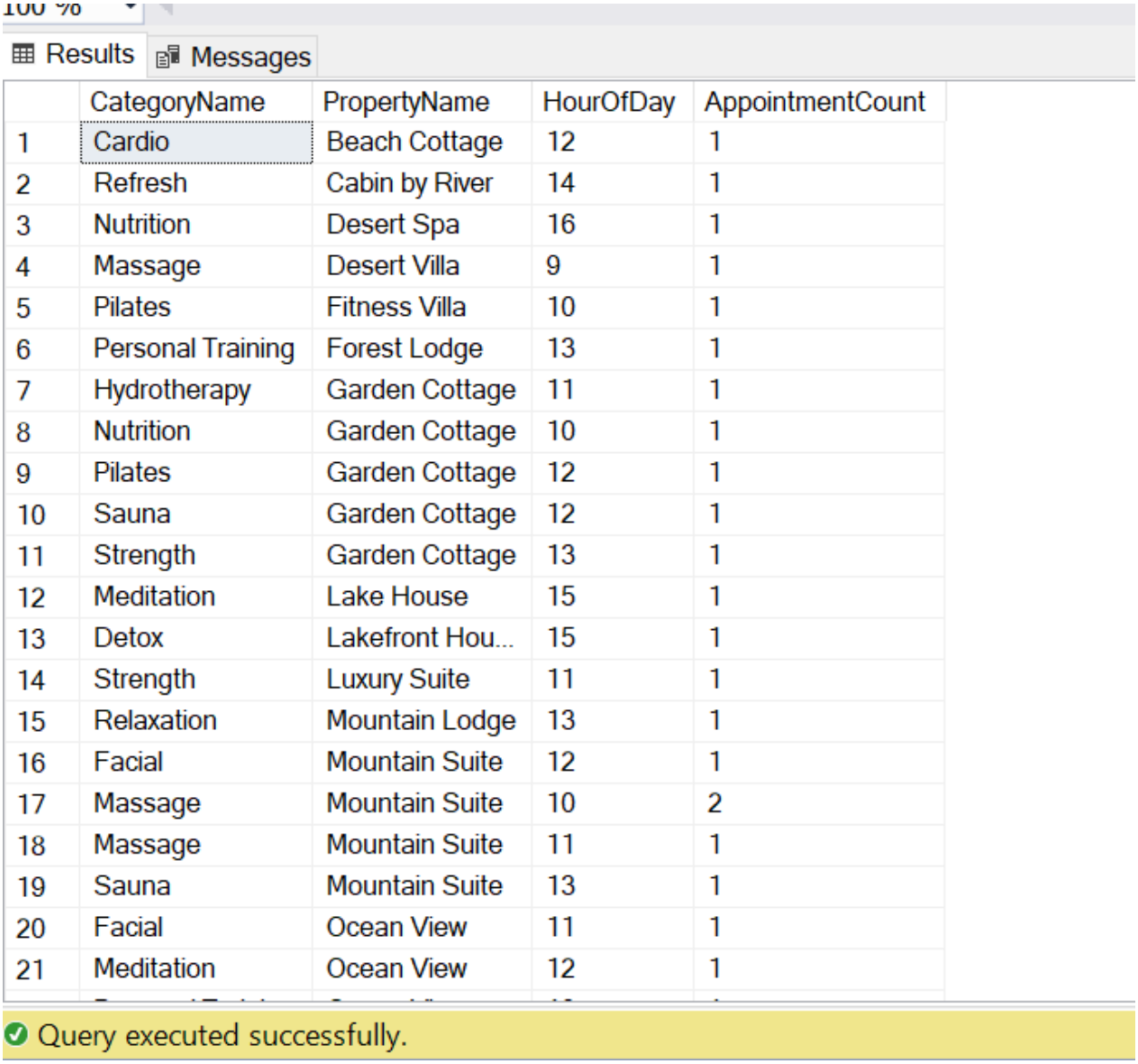
-- and extracted hour of ScheduledDateTime.

(b) CREATE VIEW Statement

```
CREATE VIEW service.vw_ServiceUtilizationByHour AS
SELECT
  sc.CategoryName,
  prop.PropertyName,
  DATEPART(HOUR, sa.ScheduledDateTime)     AS HourOfDay,
  COUNT(sa.AppointmentID)                  AS AppointmentCount
    FROM service.SERVICEAPPOINTMENT  AS sa
    INNER JOIN booking.STAY                AS s    ON sa.StayID     =
s.StayID
```

```
    INNER JOIN property.PROPERTY           AS prop ON s.PropertyID  =
prop.PropertyID
    INNER JOIN service.SERVICE             AS svc  ON sa.ServiceID  =
svc.ServiceID
    INNER JOIN service.SERVICECATEGORY     AS sc   ON svc.CategoryID =
sc.CategoryID
GROUP BY
  sc.CategoryName,
  prop.PropertyName,
  DATEPART(HOUR, sa.ScheduledDateTime);
GO


-- Sample query — find peak hours for each category at each property,
-- ordered to surface the busiest slots first:
SELECT *
FROM service.vw_ServiceUtilizationByHour
ORDER BY PropertyName, CategoryName, AppointmentCount DESC;
```



Results | Messages

	CategoryName	PropertyName	HourOfDay	AppointmentCount
1	Cardio	Beach Cottage	12	1
2	Refresh	Cabin by River	14	1
3	Nutrition	Desert Spa	16	1
4	Massage	Desert Villa	9	1
5	Pilates	Fitness Villa	10	1
6	Personal Training	Forest Lodge	13	1
7	Hydrotherapy	Garden Cottage	11	1
8	Nutrition	Garden Cottage	10	1
9	Pilates	Garden Cottage	12	1
10	Sauna	Garden Cottage	12	1
11	Strength	Garden Cottage	13	1
12	Meditation	Lake House	15	1
13	Detox	Lakefront Hou...	15	1
14	Strength	Luxury Suite	11	1
15	Relaxation	Mountain Lodge	13	1
16	Facial	Mountain Suite	12	1
17	Massage	Mountain Suite	10	2
18	Massage	Mountain Suite	11	1
19	Sauna	Mountain Suite	13	1
20	Facial	Ocean View	11	1
21	Meditation	Ocean View	12	1

Query executed successfully.

Figure 9: Showing VIEW 2 ANALYTICAL

Interpretation of Output (View 2 – vw_ServiceUtilizationByHour)

The result set shows the number of service appointments grouped by service category, property, and hour of day. Each row represents how many appointments were scheduled for a specific service category at a particular property during a given hour.

For example:

- **Massage at Mountain Suite at 10:00** shows an AppointmentCount of 2, indicating this is a relatively higher-demand time slot.
- **Massage at Desert Villa at 9:00** shows 1 appointment.
- **Pilates at Fitness Villa at 10:00** shows 1 appointment.
- Multiple categories at Garden Cottage (Hydrotherapy, Nutrition, Pilates, Sauna, Strength) appear across different hours, each with 1 appointment.

The HourOfDay column is extracted using DATEPART(HOUR, ScheduledDateTime), which allows the system to analyze demand patterns by time rather than by full timestamp.

Contribution Statement – Milestone 2

Suggested Role	Primary Owner	Core Responsibilities
Team Coordinator	**Sharon Wamalwa (Member 2)**	• Scheduled team meetings and maintained the shared task tracker. • Ensured the Revision Summary section was completed before beginning Milestone 2 work. • Compiled all sections into the final Word document. • Verified that every required rubric element was present prior to submission. • Created and tested the final database backup (.bak) file and attached it to the submission. • Cross-reviewed at least one CREATE TABLE script and one SQL query for correctness. • Confirmed that the database backup restored cleanly in SSMS.

		• Applied ER-to-relational mapping rules (Chapter 3, Jukic et al.) to produce the complete relational schema in standard notation.
Schema Architect &Normalization Lead	**Abigael Torome (Member 3)**	• Enumerated all non-trivial functional dependencies for every relation. • Demonstrated 3NF compliance (and BCNF where applicable) with written justifications. • Documented any lossless, dependency-preserving decompositions required to eliminate 2NF or 3NF violations. • Produced the Revision Summary documenting all changes from Milestone 1 feedback.
DDL Developer & Constraints Specialist	**Isaac Wainaina (Member 4)**	• Cross-reviewed DDL scripts to confirm that table structures matched the normalized schema. • Validated that all foreign key references were consistent with the relational schema.

		• Wrote all CREATE TABLE statements with correct column names, data types, NOT NULL, UNIQUE, DEFAULT, and CHECK constraints. • Implemented all PRIMARY KEY and FOREIGN KEY constraints with appropriate ON DELETE / ON UPDATE rules, with written justification for each choice. • Created required non-clustered indexes with justification. • Created View 1 (vw_GuestStayDetail) and View 2 (vw_ServiceUtilizationByHour). • Documented the DDL script and explained each constraint decision in the report. • Implemented structural ER changes while creating the physical database, including introducing an associative entity between Reservation and Questionnaire tables and revising the Membership–Guest relationship to include a partial key and identifying relationship. • Verified that sample data populated without referential integrity errors.
Data Engineer & Query Developer A	**Swetha Lnu (Member 5)**	• Generated and validated all sample data (minimum 15 rows per core table), ensuring domain consistency and referential integrity. • Wrote and tested Queries 1–3 (Multi-Table JOIN, Aggregate + GROUP BY/HAVING, CTE for top services). • Provided SSMS screenshots and plain-English interpretations for each query. • Documented the data generation approach and applied domain-specific business rules.

		• Cross-reviewed Queries 4–6 and validated the stored procedure logic before submission. • Verified that sample data returned meaningful, non-trivial results for all queries.
Query Developer & Procedures Specialist	**Sanjay Singh (Member 1)**	• Wrote and tested Queries 4–6 (Window Function, Correlated Subquery, and DML via stored procedure). • Built and tested stored procedure usp_BookServiceAppointment, including validation checks and RAISERROR calls. • Demonstrated one successful and one intentionally failing procedure execution with SSMS screenshots. • Provided business interpretations for each query result. • Cross-reviewed Queries 1–3 for logical correctness. • Tested the stored procedure against populated sample data to confirm correct behavior and constraint enforcement.

MILESTONE 3

Milestone 2 Revision Summary

GUEST.ReservationID FK is backwards

The ReservationID has been removed from the GUEST table and added GuestID as a foreign key on RESERVATION so that a guest can exist without having to make any reservation.

ACTIVITY.StayID FK makes activities stay-specific

StayID has been removed from ACTIVITY and activity is linked to property using PropertyID to activity is linked to property instead of a stay. ACTIVITYATTENDANCE acts as a bridge between guests and activities. ACTIVITY to PROPERTY has an optional to many relationships which allows multiple guests to attend an activity within a property.

ACTIVITYATTENDANCE is 1:1 with ACTIVITY

ActivityID has been removed from ACTIVITYATTENDANCE as the primary key to allow multiple attendees to a particular activity. The new primary key is a composite attribute consisting of ACTIVITYID and GUESTID.

No DELIVERS bridge table

The DELIVERS table has been added as a link between STAFF and SERVICES. The table includes StaffID, ServiceID with composite attributes of StaffID and ServiceID as primary key and StaffID and foreign key.

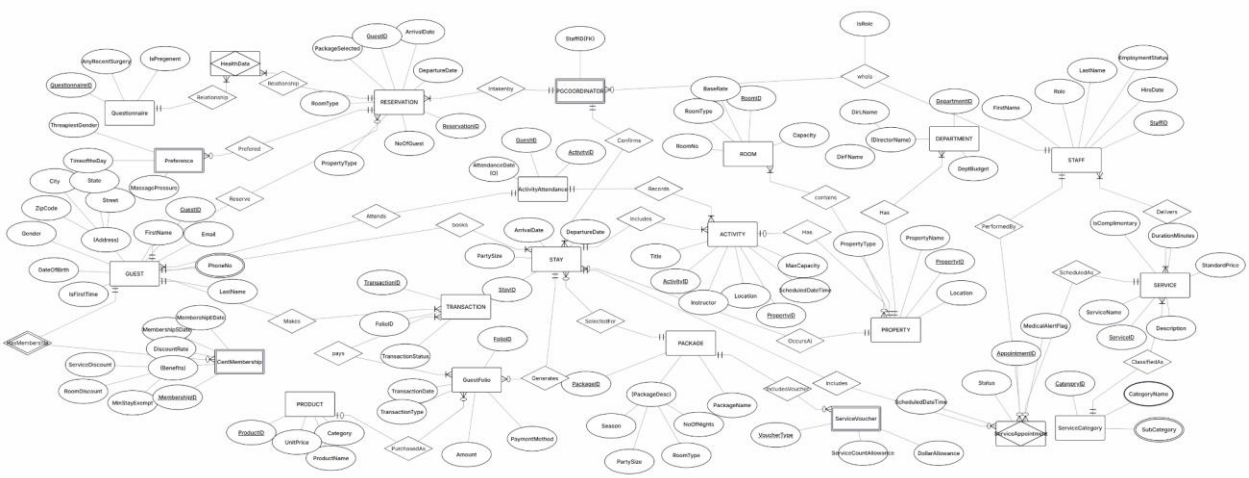


Figure 2: Corrected ERD

Stored Procedure — Missing Validation

The DELIVERS table provides information on staff members who qualify to deliver a specific service. The procedures demonstrate one successful and one failed call. The failed call shows that the staff is unqualified to deliver the service

Successful call:

```
Service appointment booked successfully.

Completion time: 2026-04-25T21:38:30.1064079-05:00
```

Failed call: Failed to book an appointment

```
Msg 50000, Level 16, State 1, Procedure service.usp_BookServiceAppointment, Line 50 [Batch Start Line 280]
Staff is not qualified to deliver this service.

Completion time: 2026-04-25T21:36:12.4538292-05:00
```

Query Mismatches — Three Queries Need Correction

Query 1

The query has been rewritten to answer the question and includes guest name, property, arrival date, departure date package name, total charges and name of assigned program coordinator for each stay.

```
use CanyonRanchDB

/* Query 1: Multi-Table JOIN */

SELECT

      g.FirstName + ' ' + g.LastName AS GuestName,

      p.PropertyName,

      s.ArrivalDate,

      s.DepartureDate,

      pk.PackageName,

      SUM(f.Amount) AS TotalCharges,

      st.FirstName + ' ' + st.LastName AS ProgramCoordinator

FROM booking.STAY s

JOIN booking.GUEST g

      ON s.GuestID = g.GuestID

JOIN property.PROPERTY p

      ON s.PropertyID = p.PropertyID

JOIN package.PACKAGE pk

      ON s.PackageID = pk.PackageID

JOIN staff.STAFF st

      ON s.StaffID = st.StaffID
```

```
LEFT JOIN billing.GUESTFOLIO f
      ON s.StayID = f.StayID
GROUP BY
      g.FirstName, g.LastName,
      p.PropertyName,
      s.ArrivalDate,
      s.DepartureDate,
      pk.PackageName,
      st.FirstName, st.LastName
ORDER BY s.ArrivalDate;
```

Query 2

The query has been updated to filter revenue by service category exceeding $10000.

```
/* Query 2: Aggregate + GROUP BY + HAVING */
SELECT
      p.PropertyName,
      sc.CategoryName,
      COUNT(DISTINCT st.GuestID) AS TotalGuests,
      SUM(sv.StandardPrice) AS TotalRevenue,
      AVG(sv.StandardPrice) AS AvgRevenuePerGuest
FROM booking.STAY st
JOIN property.PROPERTY p
      ON st.PropertyID = p.PropertyID
JOIN service.SERVICEAPPOINTMENT sa
      ON st.StayID = sa.StayID
JOIN service.SERVICE sv
```

```
      ON sa.ServiceID = sv.ServiceID
JOIN service.SERVICECATEGORY sc
      ON sv.CategoryID = sc.CategoryID
WHERE p.PropertyID = 'P001'
GROUP BY
      p.PropertyName,
      sc.CategoryName
HAVING SUM(sv.StandardPrice) > 10000;
```

Query 3

The query uses CTE to join SERVICEAPPOINTMENT to SERVICE and PROPERTY, identifying top 5 most frequently booked services in each property.

```
/* Query 3: CTE + Ranking */
WITH ServiceBookings AS (
      SELECT
      p.PropertyID,
      p.PropertyName,
      s.ServiceID,
      s.ServiceName,
      sc.CategoryName,
      COUNT(sa.ServiceID) AS TotalBookings,
      SUM(s.StandardPrice) AS TotalRevenue,
      ROW_NUMBER() OVER (
            PARTITION BY p.PropertyID
            ORDER BY COUNT(sa.ServiceID) DESC
      ) AS rn
```

```
	FROM property.PROPERTY p
	JOIN booking.STAY st
	ON p.PropertyID = st.PropertyID
	JOIN service.SERVICEAPPOINTMENT sa
	ON st.StayID = sa.StayID
	JOIN service.SERVICE s
	ON sa.ServiceID = s.ServiceID
	JOIN service.SERVICECATEGORY sc
	ON s.CategoryID = sc.CategoryID
	GROUP BY
	p.PropertyID,
	p.PropertyName,
	s.ServiceID,
	s.ServiceName,
	sc.CategoryName
)
SELECT
	PropertyName,
	ServiceName,
	CategoryName,
	TotalBookings,
	TotalRevenue
FROM ServiceBookings
WHERE rn <= 5
ORDER BY PropertyName, TotalBookings DESC;
```

Query 5

The STAY table includes two stays for the same guest at the same property at different dates making it possible to calculate increased spending from the previous stay and applying the 20% filter.



	StaffID	StaffFirstName	StaffLastName	GuestID	GuestFirstName	GuestLastName	TotalAppointments
1	S001	Alice	Miller	G001	Adam	Smith	7
2	S003	Carol	Anderson	G001	Adam	Smith	2
3	S005	Eva	Moore	G005	Ethan	Harris	3
4	S009	Ivy	Martin	G003	Charlie	Brown	3

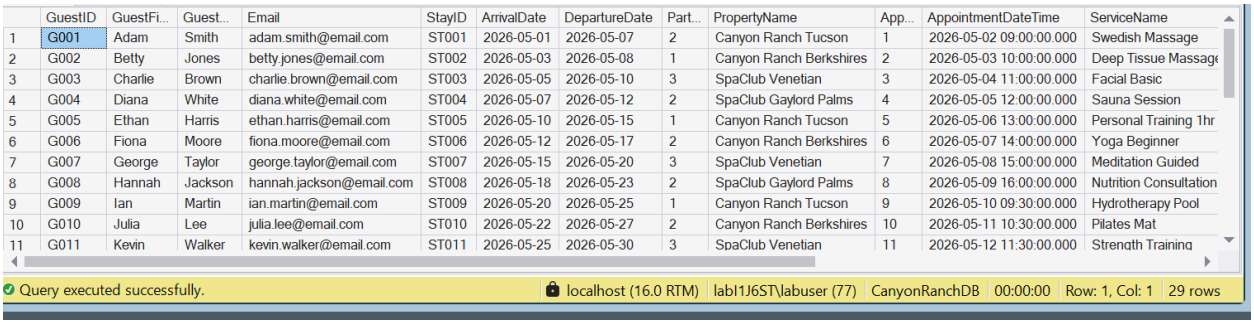
Views

View 1 uses left join to include all guests with or without previous records on stays and appointments.

```
CREATE VIEW booking.vw_GuestStayDetail AS
SELECT
  g.GuestID,
  g.FirstName                                AS GuestFirstName,
  g.LastName                                 AS GuestLastName,
  g.Email,
  s.StayID,
  s.ArrivalDate,
  s.DepartureDate,
  s.PartySize,
 prop.PropertyName,
  sa.AppointmentID,
  sa.ScheduledDateTime                       AS AppointmentDateTime,
  svc.ServiceName,
  sa.Status                                  AS AppointmentStatus,
  sa.MedicalAlertFlag,
```

```
  pref.MassagePressure,
  pref.TherapistGender,
  pref.TimeOfDay
    FROM booking.GUEST                      AS g
      LEFT JOIN booking.RESERVATION         AS r    ON r.GuestID       =
g.GuestID
    LEFT JOIN booking.PREFERENCE            AS pref ON r.ReservationID =
pref.ReservationID
    LEFT JOIN booking.STAY                  AS s    ON g.GuestID       =
s.GuestID
    LEFT JOIN property.PROPERTY             AS prop ON s.PropertyID    =
prop.PropertyID
    LEFT JOIN service.SERVICEAPPOINTMENT    AS sa   ON s.StayID        =
sa.StayID
    LEFT JOIN service.SERVICE               AS svc  ON sa.ServiceID    =
svc.ServiceID;
GO


-- Sample query — retrieve full detail for a specific guest's most recent
stay:
-- SELECT *
-- FROM booking.vw_GuestStayDetail
-- WHERE GuestID = N'G000001'
-- ORDER BY ArrivalDate DESC;
```



	GuestID	GuestFi...	Guest...	Email	StayID	ArrivalDate	DepartureDate	Part...	PropertyName	App...	AppointmentDateTime	ServiceName
1	G001	Adam	Smith	adam.smith@email.com	ST001	2026-05-01	2026-05-07	2	Canyon Ranch Tucson	1	2026-05-02 09:00:00.000	Swedish Massage
2	G002	Betty	Jones	betty.jones@email.com	ST002	2026-05-03	2026-05-08	1	Canyon Ranch Berkshires	2	2026-05-03 10:00:00.000	Deep Tissue Massage
3	G003	Charlie	Brown	charlie.brown@email.com	ST003	2026-05-05	2026-05-10	3	SpaClub Venetian	3	2026-05-04 11:00:00.000	Facial Basic
4	G004	Diana	White	diana.white@email.com	ST004	2026-05-07	2026-05-12	2	SpaClub Gaylord Palms	4	2026-05-05 12:00:00.000	Sauna Session
5	G005	Ethan	Harris	ethan.harris@email.com	ST005	2026-05-10	2026-05-15	1	Canyon Ranch Tucson	5	2026-05-06 13:00:00.000	Personal Training 1hr
6	G006	Fiona	Moore	fiona.moore@email.com	ST006	2026-05-12	2026-05-17	2	Canyon Ranch Berkshires	6	2026-05-07 14:00:00.000	Yoga Beginner
7	G007	George	Taylor	george.taylor@email.com	ST007	2026-05-15	2026-05-20	3	SpaClub Venetian	7	2026-05-08 15:00:00.000	Meditation Guided
8	G008	Hannah	Jackson	hannah.jackson@email.com	ST008	2026-05-18	2026-05-23	2	SpaClub Gaylord Palms	8	2026-05-09 16:00:00.000	Nutrition Consultation
9	G009	Ian	Martin	ian.martin@email.com	ST009	2026-05-20	2026-05-25	1	Canyon Ranch Tucson	9	2026-05-10 09:30:00.000	Hydrotherapy Pool
10	G010	Julia	Lee	julia.lee@email.com	ST010	2026-05-22	2026-05-27	2	Canyon Ranch Berkshires	10	2026-05-11 10:30:00.000	Pilates Mat
11	G011	Kevin	Walker	kevin.walker@email.com	ST011	2026-05-25	2026-05-30	3	SpaClub Venetian	11	2026-05-12 11:30:00.000	Strength Training

Query executed successfully. | localhost (16.0 RTM) | lab1J6ST\labuser (77) | CanyonRanchDB | 00:00:00 | Row: 1, Col: 1 | 29 rows

Milestone 3 Abstract

The Canyon Ranch data warehouse uses a star schema with fact_ServiceAppointment as the central fact table linked to various dimension tables such as Guest, Service, Package, Date, Property, and Staff. The dimension tables are individually linked to the fact table through a many-to-one relationship, where the fact table can have multiple records corresponding to a single dimension record. The structure enables separation of measurable transactional data (fact table) from descriptive contextual data (dimensional tables) to allow for optimized query and simplified joins. The design includes service utilization, seasonal trend evaluation, and revenue analysis which are essential Canyon Ranch reporting needs. The Guest dimension uses Slowly Changing Dimension Type 2 to capture changes in customer data over time.

The data warehouse design includes a fact table and dimension table loaded with data from the CanyonRanch database. Data is extracted, transitioned, loaded into the fact and dimension tables from the Canyon Ranch database.

The report covers conceptual design, regulatory needs, data privacy risk, and ethical and governance aspects of the Canyon Ranch database system. The report discusses four levels of data sensitivity including public, internal, confidential, and restricted protected health information (PHI). Tier one is public information including property and service details and sensitive medical information is in tier four. The risks for privacy of information are unauthorized access of information, guest profile re-identification, insecure storage of financial information, inadequate access control of health data and guest preferences, and data retention.

This milestone further examines the compliance considerations under HIPAA, CCPA/CPRA and GDPR since the Canyon Ranch needs to fulfill various requirements since it collects personal, health and financial information. The ethical discussion provides two design decisions, to segregate medical information into a new table, accessible to only authorized staff and assigning row-level security for staff to only access preference records assigned to them. The governance recommendation is to consider role-based access control, column-level security, and audit trails.

DIMENSIONAL DATA WAREHOUSE CONCEPTUAL DESIGN

1. Schema Selection and Justification

A **star schema** is selected for the Canyon Ranch data warehouse due to its simplicity and efficiency in supporting analytical processing. This design consists of a central fact table connected directly to multiple dimension tables, reducing join complexity and improving query performance. Such a structure is well-suited for business intelligence operations, including aggregations, ROLLUP, CUBE, and time-series analysis.

The star schema aligns with Canyon Ranch's analytical requirements by enabling fast and flexible reporting across key business perspectives such as guests, services, properties, and time.

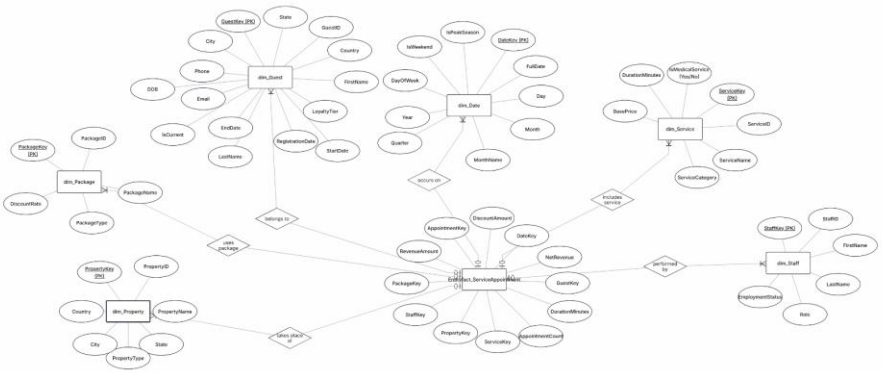


Figure 3: DW ERD Diagram

2. Fact Table Grain Definition

The grain of the fact table is defined as **one row per service appointment**. Each record represents a single service delivered to a guest at a specific property on a specific date. It

includes relevant details such as the type of service provided, the staff involved, and associated financial measures like revenue and duration.

This level of granularity supports accurate aggregation of key metrics, including total revenue, service utilization, and appointment trends across multiple dimensions such as time, guest, service, and property.

3. Slowly Changing Dimension (SCD) – Guest

Certain guest attributes are expected to change over time and must be tracked for analytical purposes. These include:

- Address (City, State, Country)
- Contact information (Email, Phone)
- Loyalty Tier

To manage these changes, the **Guest dimension is implemented using a Slowly Changing Dimension (SCD) Type 2 strategy**. Instead of overwriting existing records, a new row is created whenever a change occurs. Each version of the record is maintained using attributes such as start date, end date, and a current record indicator.

This approach preserves historical guest information, allowing the data warehouse to maintain a complete timeline of changes. As a result, analysts can track how guest characteristics evolve over time and analyze their impact on service usage and revenue patterns.

4. Data Warehouse Schema Diagram Explanation

The data warehouse is represented using a **star schema diagram**, with the central fact table, **fact_ServiceAppointment**, connected to multiple dimension tables including Date, Guest, Service, Property, Staff, and Package. Each dimension table is linked directly to the fact table through a many-to-one relationship, where multiple fact records correspond to a single dimension record.

This structure clearly separates measurable transactional data (fact table) from descriptive contextual data (dimension tables), enabling simplified joins and optimized query performance. The design effectively supports Canyon Ranch's reporting needs, including revenue analysis, service utilization, and seasonal trend evaluation.

DW PHYSICAL IMPLEMENTATION

The data warehouse design includes a fact table Fact_ServiceAppointment and six dimension tables. The surrogate keys use INT IDENTITY (1, 1) for performance and to decouple the warehouse from OLTP system's natural keys. Each dimension keeps the original business key such as GuestID and ServiceID so that records are traceable to the OLTP database if needed. Each FK references the surrogate key of its dimension, not the business key. This is the standard star schema pattern the fact table joins to dimensions via integer surrogate keys for query performance.

The dim_Guest table includes three SCD Type 2 columns which are StartDate, EndDate and IsCurrent. The dim_Date loads every calendar date from 2000-01-01 through 2010-12-31 using recursive CTE and treats Saturday as weekday 7 and Sunday as weekday 1.

ON DELETE NO ACTION / ON UPDATE NO ACTION on every FK. In a data warehouse, dimension records should never be hard-deleted because doing so would orphan historical fact rows and corrupt analytical reports. Updates to surrogate keys also should not happen, surrogate keys are stable by design. Constraints are added via ALTER TABLE rather than inline in the CREATE TABLE, which makes it easy to drop and re-add a single constraint later without rebuilding the fact table. Each constraint is named explicitly so it can be referenced directly when troubleshooting or disabling for bulk loads.

DW SPECIFIC QUERIES

Query 1: Using ROLLUP

Calculate total revenue aggregated at three levels by individual service category, by property, and by property and year combined

Part 1: by individual service category

```
SELECT
      ds.ServiceCategory,
      SUM(f.NetRevenue) AS TotalRevenue
FROM CR_DW.fact_ServiceAppointment f
JOIN CR_DW.dim_Service ds
      ON f.ServiceKey = ds.ServiceKey
GROUP BY ROLLUP (ds.ServiceCategory);
```

	ServiceCategory	TotalRevenue
1	Cardio	8500.00
2	Facial	303200.00
3	Fitness	190000.00
4	Hydrotherapy	214500.00
5	Massage	495620.00
6	Meditation	117000.00
7	Nutrition	9000.00
8	Pilates	15520.00
9	Recovery	18800.00
10	Relaxation	114000.00
11	Spa	4800.00
12	Strength	184300.00
13	Wellness	12090.00
14	Yoga	13650.00
15	NULL	1700980.00

Part 2: by property and by property and year combined with all three combination at end

```
SELECT
      dp.PropertyName,
      dd.Year,
      ds.ServiceCategory,
      SUM(fsa.NetRevenue) AS TotalRevenue
FROM CR_DW.fact_ServiceAppointment fsa
JOIN CR_DW.dim_Property dp
      ON fsa.PropertyKey = dp.PropertyKey
JOIN CR_DW.dim_Date dd
      ON fsa.DateKey = dd.DateKey
JOIN CR_DW.dim_Service ds
      ON fsa.ServiceKey = ds.ServiceKey
GROUP BY
      ROLLUP (dp.PropertyName, dd.Year, ds.ServiceCategory)
ORDER BY
      dp.PropertyName,
      dd.Year DESC;
```

	PropertyName	Year	ServiceCategory	TotalRevenue
1	NULL	NULL	NULL	1700980.00
2	Canyon Ranch Berkshires	2006	NULL	40110.00
3	Canyon Ranch Berkshires	NULL	NULL	40110.00
4	Canyon Ranch Tucson	2006	NULL	1292750.00
5	Canyon Ranch Tucson	NULL	NULL	1292750.00
6	SpaClub Gaylord Palms	2006	NULL	22300.00
7	SpaClub Gaylord Palms	NULL	NULL	22300.00
8	SpaClub Venetian	2006	NULL	345820.00
9	SpaClub Venetian	NULL	NULL	345820.00
10	SpaClub Gaylord Palms	2006	Cardio	8500.00
11	Canyon Ranch Tucson	2006	Facial	228000.00
12	SpaClub Venetian	2006	Facial	75200.00
13	Canyon Ranch Tucson	2006	Fitness	190000.00
14	Canyon Ranch Tucson	2006	Hydrotherapy	214500.00
15	Canyon Ranch Berkshires	2006	Massage	13020.00
16	Canyon Ranch Tucson	2006	Massage	482600.00
17	Canyon Ranch Tucson	2006	Meditation	57000.00
18	SpaClub Venetian	2006	Meditation	60000.00
19	SpaClub Gaylord Palms	2006	Nutrition	9000.00
20	Canyon Ranch Berkshires	2006	Pilates	8000.00
21	SpaClub Venetian	2006	Pilates	7520.00
22	SpaClub Venetian	2006	Recovery	18800.00
23	Canyon Ranch Tucson	2006	Relaxation	114000.00
24	SpaClub Gaylord Palms	2006	Spa	4800.00
25	SpaClub Venetian	2006	Strength	184300.00
26	Canyon Ranch Berkshires	2006	Wellness	12090.00
27	Canyon Ranch Berkshires	2006	Yoga	7000.00
28	Canyon Ranch Tucson	2006	Yoga	6650.00

Explanation

In the above query, rollup is used to calculate the total revenue aggregated at three levels. We have joined the relations to fetch the data as per need. In the first part, we have aggregated the total revenue based on the individual service category, and in part 2, total revenue is calculated by property, by property and year together and then at last, by each of the combination for more simplicity of revenue by property in different years in each individual services.

Query 2: Using GROUPING SETS or CUBE

Produce a summary report showing total appointment counts broken down by (a) property alone, (b) service category alone, (c) property and service category together,

and (d) the grand total

```
SELECT
      dp.PropertyName,
      ds.ServiceCategory,
      SUM(f.AppointmentCount) AS TotalAppointments
FROM CR_DW.fact_ServiceAppointment f
JOIN CR_DW.dim_Property dp
      ON f.PropertyKey = dp.PropertyKey
JOIN CR_DW.dim_Service ds
      ON f.ServiceKey = ds.ServiceKey
GROUP BY GROUPING SETS (
      (dp.PropertyName, ds.ServiceCategory),  -- detailed by property and
service category
      (dp.PropertyName),                        -- property only
      (ds.ServiceCategory),                     -- category only
      ()                                        -- grand total
)
ORDER BY
      dp.PropertyName,
      ds.ServiceCategory;
```

	PropertyName	ServiceCategory	TotalAppointments
1	NULL	NULL	29
2	NULL	Cardio	1
3	NULL	Facial	4
4	NULL	Fitness	2
5	NULL	Hydrotherapy	2
6	NULL	Massage	7
7	NULL	Meditation	2
8	NULL	Nutrition	1
9	NULL	Pilates	2
10	NULL	Recovery	1
11	NULL	Relaxation	1
12	NULL	Spa	1
13	NULL	Strength	2
14	NULL	Wellness	1
15	NULL	Yoga	2
16	Canyon Ranch Berkshires	NULL	4
17	Canyon Ranch Berkshires	Massage	1
18	Canyon Ranch Berkshires	Pilates	1
19	Canyon Ranch Berkshires	Wellness	1
20	Canyon Ranch Berkshires	Yoga	1
21	Canyon Ranch Tucson	NULL	16
22	Canyon Ranch Tucson	Facial	3
23	Canyon Ranch Tucson	Fitness	2
24	Canyon Ranch Tucson	Hydrotherapy	2
25	Canyon Ranch Tucson	Massage	6
26	Canyon Ranch Tucson	Meditation	1
27	Canyon Ranch Tucson	Relaxation	1
28	Canyon Ranch Tucson	Yoga	1
29	SpaClub Gaylord Palms	NULL	3
30	SpaClub Gaylord Palms	Cardio	1
31	SpaClub Gaylord Palms	Nutrition	1
32	SpaClub Gaylord Palms	Spa	1
33	SpaClub Venetian	NULL	6
34	SpaClub Venetian	Facial	1
35	SpaClub Venetian	Meditation	1
36	SpaClub Venetian	Pilates	1
37	SpaClub Venetian	Recovery	1
38	SpaClub Venetian	Strength	2

Explanation

In this query, GROUPING SETS is used to group by different attributes as per instructions, and then appointment is calculated. In the above output table, the appointment is calculated of grand total at first, then by services categories, followed by property and property with services.

Query 3 (Time-Series Window Function):

For each property and each calendar month, calculate the number of appointments and the 3-month rolling average using a window function. Identify months where actual appointments exceed the rolling average by more than 15% (potential seasonal peaks).

```
WITH MonthlyAppointments AS (
	SELECT
	dp.PropertyName,
	dd.Year,
	dd.Month,
	SUM(f.AppointmentCount) AS TotalAppointments
	FROM CR_DW.fact_ServiceAppointment f
	JOIN CR_DW.dim_Property dp
	ON f.PropertyKey = dp.PropertyKey
	JOIN CR_DW.dim_Date dd
	ON f.DateKey = dd.DateKey
	GROUP BY
	dp.PropertyName,
	dd.Year,
	dd.Month
)

SELECT
	PropertyName,
	Year,
	Month,
	TotalAppointments,

	AVG(TotalAppointments) OVER (
	PARTITION BY PropertyName
	ORDER BY Year, Month
	ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
	) AS RollingAvg_3Months,

	CASE
	WHEN TotalAppointments > 1.15 *
		AVG(TotalAppointments) OVER (
			PARTITION BY PropertyName
			ORDER BY Year, Month
			ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
		)
	THEN 'Peak Month'
```

```
        ELSE 'Normal'
        END AS TrendFlag

FROM MonthlyAppointments
ORDER BY PropertyName, Year, Month;
```

	PropertyName	Year	Month	TotalAppointments	RollingAvg_3Months	TrendFlag
1	Canyon Ranch Berkshires	2006	5	4	4	Normal
2	Canyon Ranch Tucson	2006	5	8	8	Normal
3	Canyon Ranch Tucson	2006	7	2	5	Normal
4	Canyon Ranch Tucson	2006	8	6	5	Peak Month
5	SpaClub Gaylord Palms	2006	5	3	3	Normal
6	SpaClub Venetian	2006	5	6	6	Normal

Explanation

In this query, we have used the time series window of 3 months rolling average and calculated the peak month trend based on the rolling average. The data output shows that Aug 2026 is the peak month trend at property CanyonRanch Tucson.

Query 4 (Business Intelligence — Canyon Ranch Specific):

Write a query of your own design that joins the fact table to at least two dimension tables and answers one of the analytical requirements from Section 3.2. State the business question clearly and explain how your result could be used by Canyon Ranch management to make a specific decision.

Question from Section 3.2: Average revenue per guest by package type, property, and visit frequency tier.

```
 WITH GuestVisitStats AS (
        SELECT
        f.GuestKey,
        f.PropertyKey,
        COUNT(*) AS TotalVisits,
        SUM(f.NetRevenue) AS TotalRevenue
        FROM CR_DW.fact_ServiceAppointment f
        GROUP BY
        f.GuestKey,
        f.PropertyKey
),

GuestTier AS (
```

```
	SELECT
	gvs.*,
	CASE
		WHEN TotalVisits >= 10 THEN 'High Frequency'
		WHEN TotalVisits BETWEEN 4 AND 9 THEN 'Medium Frequency'
		ELSE 'Low Frequency'
	END AS VisitTier
	FROM GuestVisitStats gvs
)

SELECT
	dp.PropertyName,
	VisitTier,
	COUNT(*) AS NumberOfGuests,
	AVG(TotalRevenue) AS AvgRevenuePerGuest,
	AVG(TotalVisits) AS AvgVisits
FROM GuestTier gt
JOIN CR_DW.dim_Property dp
	ON gt.PropertyKey = dp.PropertyKey
GROUP BY
	dp.PropertyName,
	VisitTier
ORDER BY
	dp.PropertyName,
	VisitTier;
```

	PropertyName	VisitTier	NumberOfGuests	AvgRevenuePerGuest	AvgVisits
1	Canyon Ranch Berkshires	Low Frequency	4	10027.500000	1
2	Canyon Ranch Tucson	High Frequency	1	910100.000000	11
3	Canyon Ranch Tucson	Low Frequency	3	127550.000000	1
4	SpaClub Gaylord Palms	Low Frequency	3	7433.333333	1
5	SpaClub Venetian	Low Frequency	4	86455.000000	1

Explanation

This query segments guests based on visit frequency at each property and calculates the average revenue generated per segment. It reveals how customer value changes with engagement level. High-frequency guests typically contribute significantly higher revenue, allowing Canyon Ranch to prioritize loyalty programs and targeted retention strategies for this segment.

DATA SENSITIVITY CLASSIFICATION

Canyon Ranch's database stores a wide spectrum of data ranging from general operational information to highly sensitive personal health records. To ensure appropriate protection and access control, all tables and attribute groups have been classified into four sensitivity tiers:

Public, Internal, Confidential, and Restricted/PHI. This classification follows the principle that data should be protected in proportion to the harm its unauthorized disclosure could cause.

Tier 1 : Public

Public data carries minimal privacy risk and could be disclosed without harm to any individual. In the Canyon Ranch database, this tier includes general property information stored in the PROPERTY table such as PropertyName, Location, and PropertyType, as well as service catalog information in the SERVICE and SERVICECATEGORY tables including ServiceName, CategoryName, DurationMinutes, StandardPrice, and IsComplimentary. This information mirrors what Canyon Ranch already publishes on its website and in guest brochures and poses no risk if accessed broadly.

Tier 2 : Internal

Internal data is operational information intended for staff use that should not be disclosed outside the organization but does not directly identify or harm guests if accessed by authorized personnel. This tier includes the STAFF, PGCOORDINATOR, DEPARTMENT, ROOM, PACKAGE, SERVICEVOUCHER, SERVICECATEGORY_SUBCATEGORY, DELIVERS, PRODUCTS, and TRANSACTION tables. Staff employment records such as HireDate, EmploymentStatus, and Role are sensitive enough to restrict from public view but are routinely accessed by management and HR. Package pricing, room rates, department budgets, and product catalogs fall into this tier as proprietary business information that requires internal access controls.

Tier 3 : Confidential

Confidential data contains personally identifiable information (PII) about guests that, if disclosed, could cause reputational harm, financial loss, or identity theft. This tier includes the GUEST table attributes such as FirstName, LastName, DateOfBirth, Gender, Email, Street, City, State, and ZipCode, as well as the GUEST_PHONENO, RESERVATION, CENTMEMBERSHIP, STAY, GUESTFOLIO, PAYS, ACTIVITYATTENDANCE, and ACTIVITY tables. Guest contact information, home addresses, visit history, financial transactions, and membership discount details are all highly sensitive personal data subject to privacy regulations. The GUESTFOLIO table in particular contains financial transaction records including PaymentMethod and Amount that must be protected against unauthorized access and potential fraud.

Tier 4 : Restricted / PHI (Protected Health Information)

This is the most sensitive tier in the Canyon Ranch database and requires the strongest access controls, encryption, and audit logging. It includes all data that relates to guests' physical and mental health status. Specifically this tier covers the QUESTIONNAIRE table (AnyRecentSurgery, IsPregnant), the HEALTHDATA associative table linking questionnaires to reservations, the PREFERENCE table (MassagePressure, TherapistGender, TimeOfDay) which captures medically relevant treatment preferences, and the MedicalAlertFlag column in the SERVICEAPPOINTMENT table. Canyon Ranch's Health and Healing department collects information about conditions such as pregnancy, recent surgery, eating disorders, behavioral health, and chronic pain data that is arguably equivalent to Protected Health Information under

HIPAA. Unauthorized disclosure of this data could cause severe harm including discrimination, insurance consequences, and profound violations of guest privacy and trust.

FIVE PRIVACY RISK SCENARIOS

Privacy Risk Scenarios

Canyon Ranch collects some of the most sensitive personal data imaginable in a hospitality context. The following five privacy risk scenarios identify concrete threats arising specifically from the data stored in the CanyonRanchDB database, the potential harm each poses to guests, and recommended mitigations grounded in current technical and procedural best practices.

Risk 1 Unauthorized Access to Health and Medical Data

Threat Scenario

A malicious insider such as a front-desk staff member or junior IT technician with broad database access queries the QUESTIONNAIRE, HEALTHDATA, and SERVICEAPPOINTMENT tables to retrieve guest medical flags including pregnancy status, recent surgery history, and MedicalAlertFlag values. Because the database currently grants schema-level access rather than column-level access, any staff member with SELECT permissions on the booking or service schema can retrieve this restricted health data alongside routine operational data.

Potential Harm

Exposure of a guest's pregnancy status, surgical history, or behavioral health conditions could cause severe personal harm including discrimination in employment or insurance if the information were shared externally. For Canyon Ranch's affluent clientele, such a breach could

also result in significant reputational damage, legal liability, and loss of guest trust — directly undermining the brand's core promise of a safe and confidential wellness environment.

Recommended Mitigation

Implement column-level security on the AnyRecentSurgery, IsPregnant, and MedicalAlertFlag columns, restricting SELECT access to only authorized medical staff and program coordinators. Additionally, apply row-level security policies so that staff can only access health records associated with guests currently under their care.

Risk 2 Guest Profile Re-identification Through Combined Data

Threat Scenario

Although no single table in CanyonRanchDB contains a complete guest profile, an attacker or unauthorized employee who joins the GUEST, PREFERENCE, STAY, SERVICEAPPOINTMENT, and QUESTIONNAIRE tables can construct a highly detailed personal dossier including name, home address, health conditions, treatment preferences, therapist gender preference, and visit history for any guest in the database. This re-identification risk is particularly acute because Canyon Ranch serves a small, identifiable, and high-profile guest population of approximately 17,000 guests per resort per year.

Potential Harm:

A reconstructed guest profile could be used for targeted social engineering, blackmail, or identity theft. For high-profile guests including executives, celebrities, or public figures who visit Canyon Ranch for confidential health and wellness treatment, exposure of a combined profile could cause irreparable reputational and personal harm.

Recommended Mitigation

Enforce strict role-based access control (RBAC) so that no single database role has SELECT permissions across all schemas simultaneously. Program coordinators should access only guest-facing operational data, medical staff should access only health-related tables, and billing staff should access only financial tables. Cross-schema joins should be restricted to privileged reporting roles with full audit logging enabled.

Risk 3 Insecure Storage of Financial and Payment Data

Threat Scenario

The GUESTFOLIO table stores PaymentMethod and financial transaction amounts for every guest stay, and the TRANSACTION and PAYS tables record payment settlement history linked directly to GuestID. If the database server is compromised through a SQL injection attack or an unpatched vulnerability, an attacker could extract complete financial records for all guests including payment method types and total spending amounts across multiple stays.

Potential Harm

Exposure of payment method data and spending patterns could facilitate financial fraud and identity theft. Given that Canyon Ranch guests spend between $2,000 and $2,500 per stay on average and many are repeat high-net-worth visitors, the financial harm from such a breach could be substantial. Additionally, Canyon Ranch could face significant regulatory penalties and civil liability under the Payment Card Industry Data Security Standard (PCI-DSS) if cardholder data is found to be inadequately protected.

Recommended Mitigation

Payment method details beyond a tokenized reference should never be stored in the relational database. Implement Transparent Data Encryption (TDE) on the CanyonRanchDB database to protect data at rest, and ensure all connections to the database server use TLS encryption to protect data in transit. Full payment card numbers must never be stored in any table under any circumstances.

Risk 4 Inadequate Access Controls on Guest Preference and Behavioral Health Data

Threat Scenario

The PREFERENCE table stores medically and personally sensitive attributes including TherapistGender, MassagePressure, and TimeOfDay preferences collected during the pre-arrival reservation call. In the current schema, this table is accessible to any role with access to the booking schema. A program coordinator, administrative assistant, or temporary staff member could access these preference records for guests they are not assigned to, including preferences that reflect sensitive personal circumstances such as a guest's trauma history or gender identity concerns that informed their therapist preference.

Potential Harm

Exposure of guest preferences, particularly therapist gender preferences that may reflect past trauma or abuse, constitutes a serious violation of personal privacy. Canyon Ranch's own case study acknowledges that guests share deeply personal information — including histories of eating disorders and behavioral health conditions — with program coordinators. Unauthorized

access to this data by staff who have no legitimate need for it violates the principle of data minimization and could cause significant emotional harm to affected guests.

Recommended Mitigation

Restrict access to the PREFERENCE table to only the assigned program coordinator for each reservation, enforced through row-level security policies tied to the StaffID foreign key on the RESERVATION table. Implement an audit trigger on the PREFERENCE table that logs every SELECT, UPDATE, and DELETE operation including the database user, timestamp, and affected ReservationID.

Risk 5 Data Retention and Right to Erasure Non-Compliance

Threat Scenario

The CanyonRanchDB database currently has no data retention policy or archival strategy. Guest records including health questionnaires, medical alert flags, preference data, and full stay histories are retained indefinitely in active operational tables. A guest who visited Canyon Ranch years ago and wishes to have their personal data deleted — exercising their right to erasure under applicable privacy regulations — has no mechanism through which Canyon Ranch can fulfill this request without manually identifying and deleting records across at least ten interconnected tables, risking referential integrity violations due to the cascade constraints defined in the schema.

Potential Harm

Indefinite retention of sensitive health, behavioral, and financial data increases the blast radius of any future data breach. It also exposes Canyon Ranch to regulatory penalties under CCPA and

GDPR if guests invoke their right to erasure and Canyon Ranch cannot demonstrate timely and complete deletion of personal data. Guests who trusted Canyon Ranch with sensitive health information during a stay years ago may suffer significant harm if that data is later exposed in a breach.

Recommended Mitigation

Implement a formal data retention policy specifying maximum retention periods for each data category for example, seven years for financial folio records (for tax compliance), three years for operational stay records, and one year for health questionnaire data after the guest's last visit. Develop a data subject erasure procedure that anonymizes rather than hard-deletes guest records where referential integrity constraints prevent full deletion, replacing PII fields with null or anonymized values while preserving aggregate operational data.

REGULATORY ANALYSIS (HIPAA, CCPA, GDPR)

Regulatory Environment

If the CanyonRanchDB database were implemented and operated today in 2025–2026, Canyon Ranch would operate within a complex and overlapping regulatory landscape governing the collection, storage, processing, and deletion of personal and health-related data. The following analysis addresses the three most directly applicable regulatory frameworks: the Health Insurance Portability and Accountability Act (HIPAA), the California Consumer Privacy Act (CCPA) as amended by the California Privacy Rights Act (CPRA), and the General Data Protection Regulation (GDPR).

HIPAA : Health Insurance Portability and Accountability Act

HIPAA applies to covered entities defined as healthcare providers, health plans, and healthcare clearinghouses and their business associates (U.S. Department of Health and Human Services [HHS], 2025). The central question for Canyon Ranch is whether the health and healing data stored in CanyonRanchDB constitutes Protected Health Information (PHI) under HIPAA's Privacy Rule.

Canyon Ranch's Health and Healing department employs licensed medical doctors, nutritionists, exercise physiologists, and behavioral health specialists who deliver medical consultations, acupuncture, cardiac stress tests, and behavioral health services to guests (Applegate & Piccoli, 2004). The data captured in the QUESTIONNAIRE table (pregnancy status, recent surgery history), the HEALTHDATA table, and the MedicalAlertFlag column in SERVICEAPPOINTMENT directly reflects individually identifiable health information created or received by healthcare providers in the course of delivering health services. This strongly suggests that at least a portion of Canyon Ranch's Health and Healing data meets the HIPAA definition of PHI (HHS, 2025).

However, Canyon Ranch's classification as a covered entity under HIPAA is not straightforward. The company operates primarily as a hospitality and wellness business rather than a traditional healthcare provider, and guests pay directly for services rather than through health insurance. Canyon Ranch should obtain a formal legal opinion on its HIPAA covered entity status and, as a precautionary measure, implement HIPAA-compliant safeguards for all health-related data including administrative safeguards such as workforce training and access management, physical safeguards for server infrastructure, and technical safeguards including encryption and audit

controls (HHS, 2024). This precautionary approach is particularly warranted given that the Federal Trade Commission has in recent years expanded its enforcement of health data privacy beyond traditional HIPAA covered entities under its Health Breach Notification Rule, which was significantly strengthened in 2024 (Federal Trade Commission [FTC], 2024).

CCPA / CPRA : California Consumer Privacy Act and California Privacy Rights Act

The CCPA as amended by the CPRA is directly applicable to Canyon Ranch's Tucson, Arizona resort because it covers businesses that collect personal information from California residents regardless of where the business is physically located (California Attorney General, 2024). Given that Canyon Ranch's primary feeder markets include the New York metro area and the eastern corridor, a significant proportion of its guest population likely includes California residents who are entitled to CCPA/CPRA protections when they visit the Tucson property (Applegate & Piccoli, 2004).

Under the CCPA/CPRA framework as it stands in 2025–2026, Canyon Ranch would be classified as a business subject to the law if it meets any one of three thresholds: annual gross revenues exceeding $25 million, buying or selling personal information of 100,000 or more consumers annually, or deriving 50 percent or more of annual revenue from selling personal information (California Privacy Protection Agency [CPPA], 2023). Given Canyon Ranch's scale of approximately 17,000 guests per resort per year across multiple properties, the revenue threshold is almost certainly met (Applegate & Piccoli, 2004).

The CCPA/CPRA grants California resident guests several enforceable rights with direct implications for the CanyonRanchDB database design (CPPA, 2023). The right to know requires Canyon Ranch to disclose what personal data it collects, the purposes for which it is used, and

with whom it is shared. The right to delete requires Canyon Ranch to delete a guest's personal data upon request, a process complicated by the cascade constraints and referential integrity rules built into the schema as discussed in Risk 5 above. The right to correct requires Canyon Ranch to rectify inaccurate personal data, which requires an auditable update mechanism across the GUEST, PREFERENCE, and QUESTIONNAIRE tables. Critically, the CPRA introduced a new category of sensitive personal information (SPI) that includes health and medical data, precise geolocation, and financial account information all of which are present in CanyonRanchDB and grants consumers the right to limit the use and disclosure of SPI to only what is necessary to provide the requested service (California Attorney General, 2024).

GDPR : General Data Protection Regulation

Although Canyon Ranch currently operates exclusively within the United States, the GDPR becomes applicable if Canyon Ranch were to accept international guests from European Union member states. Under the GDPR's extraterritorial scope established in Article 3, any organization that offers goods or services to EU data subjects or monitors their behavior is subject to the regulation regardless of where the organization is based (European Parliament & Council of the European Union, 2016).

The GDPR's core principles have direct implications for the CanyonRanch design. The principle of lawfulness, fairness, and transparency requires that Canyon Ranch identify a valid legal basis for processing each category of personal data for routine guest contact data this would likely be contract performance, while for health data it would require explicit consent under Article 9 which governs special categories of data (European Parliament & Council of the European Union, 2016). Health data is explicitly classified as a special category under the GDPR and

requires a higher standard of protection including explicit opt-in consent, data protection impact assessments (DPIAs), and in many cases the appointment of a Data Protection Officer (DPO).

The principle of data minimization under Article 5(1)(c) requires that only data that is strictly necessary for the specified purpose be collected and stored (European Parliament & Council of the European Union, 2016). Reviewing the CanyonRanchDB schema against this principle raises questions about whether all attributes currently stored such as full home address, date of birth, and detailed preference notes are strictly necessary for delivering the Canyon Ranch experience. The principle of storage limitation under Article 5(1)(e) reinforces the data retention concerns raised in Risk 5, requiring that personal data not be kept longer than necessary for the purpose for which it was collected. Finally, the GDPR's right to erasure under Article 17 the so-called right to be forgotten would require Canyon Ranch to implement the erasure procedures described in the Risk 5 mitigation above, including the ability to demonstrate compliance through audit logs (European Parliament & Council of the European Union, 2016)

ETHICAL DESIGN DECISIONS

Database design is never a purely technical exercise every structural decision carries ethical implications for the people whose data the system stores. The following reflects on two significant ethical design decisions made during the development of CanyonRanchDB, examining the tradeoffs involved and what changes would be required to align the design more fully with principles of privacy by design, data minimization, and the responsible stewardship of sensitive guest information.

Ethical Design Decision 1

Storing MedicalAlertFlag in the SERVICEAPPOINTMENT Table

One of the most consequential design decisions in CanyonRanchDB was the placement of the MedicalAlertFlag column directly within the SERVICEAPPOINTMENT table alongside routine operational data such as ScheduledDateTime, Status, StaffID, and ServiceID. This design choice was made for operational convenience program coordinators and therapists need to see medical alerts at the point of service delivery to ensure guest safety. However, this convenience comes at a significant ethical cost.

By placing MedicalAlertFlag in the same table as appointment scheduling data, the current design grants any staff member with access to the SERVICEAPPOINTMENT table including administrative staff who manage scheduling but have no clinical role the ability to see that a guest has a medical alert associated with their appointment. The flag itself reveals that something medically significant applies to this guest, even if the underlying condition is not explicitly named. For guests who visited Canyon Ranch for sensitive behavioral health services, eating disorder treatment, or pregnancy-related wellness services, even the presence of a flag without further detail constitutes a disclosure of sensitive health information to personnel who have no legitimate clinical need for it.

A more ethically sound design would segregate medical alert data into a separate restricted table for example, a MEDICALALERT table within a dedicated health schema accessible only to authorized clinical staff and program coordinators assigned to the guest. The SERVICEAPPOINTMENT table would retain only a non-descriptive indicator that a coordinator check is required before the appointment proceeds, with the actual alert details accessible only

through the restricted table by authorized personnel. This separation implements the principle of need-to-know access at the schema level rather than relying entirely on application-layer controls, and reduces the risk of inadvertent disclosure through broad database access grants (Jukic et al., 2026).

Ethical Design Decision 2

Accessibility of Guest Preference Data Across Staff Roles

The PREFERENCE table stores personal and medically relevant guest preferences including TherapistGender, MassagePressure, and TimeOfDay, collected during the confidential pre-arrival reservation call. These preferences are linked to a ReservationID and are accessible to any staff role with SELECT permissions on the booking schema. The current design does not restrict preference data to the specific program coordinator assigned to the guest's reservation, nor does it distinguish between preferences that are purely logistical such as preferred time of day and those that may reflect sensitive personal circumstances, such as a therapist gender preference that may be rooted in past trauma or abuse.

The ethical tension here lies between operational efficiency and privacy. Canyon Ranch's program coordinators genuinely need access to guest preferences to plan and personalize the guest experience this is central to the brand's value proposition of deeply personalized wellness. However, the case study itself acknowledges that guests share highly sensitive personal information with program coordinators with an expectation of strict confidentiality (Applegate & Piccoli, 2004). The current schema does not technically enforce this confidentiality it relies entirely on staff professionalism and organizational policy rather than database-level access controls.

To implement data minimization and the principle of purpose limitation more rigorously, the PREFERENCE table should be restructured so that access is enforced at the row level through a row-level security policy tied to the StaffID of the assigned coordinator on the RESERVATION table. Staff members should only be able to retrieve preference records for guests currently assigned to them. Furthermore, the design could benefit from separating logistical preferences — time of day, massage pressure from sensitive personal preferences such as therapist gender, storing the latter in the restricted health schema alongside medical alert data. This would ensure that sensitive preference data is subject to the same elevated access controls as clinical health information, reflecting its equivalent sensitivity in the Canyon Ranch context (Jukic et al., 2026; Applegate & Piccoli, 2004).

What would need to change in the design to implement a full principle of data minimization? At minimum, Canyon Ranch should conduct a data necessity audit for every attribute in the GUEST, PREFERENCE, QUESTIONNAIRE, and SERVICEAPPOINTMENT tables, asking for each field: is this data strictly necessary to deliver the service the guest has requested, or is it being collected for convenience, historical interest, or potential future use? Fields that cannot be justified against this standard such as storing a guest's full home address when a city and state would suffice for segmentation purposes, or retaining detailed health questionnaire responses beyond the duration of the stay should be either eliminated from the schema or moved to a time-limited archive subject to automatic purging (European Parliament & Council of the European Union, 2016).

GOVERNANCE RECOMMENDATIONS

Effective data governance requires more than good schema design it demands a set of technical and procedural controls that actively enforce privacy, accountability, and security across the entire lifecycle of guest data. The following three governance recommendations are grounded in the specific tables and columns of CanyonRanchDB and are directly responsive to the privacy risks and ethical concerns identified in the preceding sections.

Recommendation 1

Implement Role-Based Access Control (RBAC) Across All Database Schemas

What it is: Role-Based Access Control is a security model in which database permissions are assigned to named roles rather than to individual user accounts, and staff members are granted only the role and therefore only the permissions required to perform their specific job function (Jukic et al., 2026).

Specific tables and columns protected*:* All tables across all eight schemas in CanyonRanchDB, with particular emphasis on the Restricted/PHI tier tables: booking.QUESTIONNAIRE, booking.HEALTHDATA, booking.PREFERENCE, and the MedicalAlertFlag column of service.SERVICEAPPOINTMENT. Additionally, confidential tier tables including booking.GUEST, booking.STAY, booking.CENTMEMBERSHIP, billing.GUESTFOLIO, and billing.TRANSACTION require role-based access restrictions.

Implementation*:* Canyon Ranch should define a minimum of five database roles reflecting its operational structure. A GuestServices role should have SELECT and UPDATE access to booking.GUEST, booking.RESERVATION, booking.STAY, and booking.PREFERENCE, but

no access to health or billing schemas. A ClinicalStaff role should have SELECT access to booking.QUESTIONNAIRE, booking.HEALTHDATA, and the MedicalAlertFlag column of service.SERVICEAPPOINTMENT, but no access to financial tables. A BillingStaff role should have SELECT and INSERT access to billing.GUESTFOLIO, billing.TRANSACTION, and billing.PAYS, but no access to health or preference data. A PropertyOperations role should have access to property.PROPERTY, property.ROOM, property.DEPARTMENT, activity.ACTIVITY, and activity.ACTIVITYATTENDANCE. Finally, a ReadOnlyReporting role should have SELECT-only access to aggregated views such as service.vw_ServiceUtilizationByHour without access to any guest-identifiable data. A DatabaseAdministrator role with full access should be restricted to a maximum of two named individuals and subject to enhanced audit logging on every action performed.

Threat mitigated*:* RBAC directly mitigates Risk 1 (unauthorized access to health data), Risk 2 (guest profile re-identification through combined data access), and Risk 4 (inadequate access controls on preference data). It also supports HIPAA's minimum necessary standard, CCPA's purpose limitation requirements, and GDPR's Article 5(1)(b) principle of purpose limitation (HHS, 2025; CPPA, 2023; European Parliament & Council of the European Union, 2016).

Recommendation 2

Implement Audit Logging via Database Triggers on Sensitive Tables

What it is*:* An audit trigger is a database trigger that automatically records every data access or modification event on a specified table into a dedicated audit log table, capturing the database user, timestamp, operation type, and affected record identifiers (Syverson & Murach, 2023).

Audit logging creates an immutable record of who accessed or modified sensitive data and when, enabling both proactive security monitoring and retrospective investigation of potential breaches.

Specific tables and columns protected*:* Audit triggers should be implemented on the following tables as a minimum: booking.QUESTIONNAIRE, booking.HEALTHDATA, booking.PREFERENCE, service.SERVICEAPPOINTMENT (specifically for INSERT, UPDATE, and SELECT operations involving MedicalAlertFlag), billing.GUESTFOLIO, billing.TRANSACTION, and booking.GUEST (for UPDATE and DELETE operations). A dedicated audit schema should be created for example, audit.ACCESS_LOG — with columns capturing EventID, TableName, OperationType, AffectedRecordID, DatabaseUser, ApplicationUser, EventTimestamp, and OldValue/NewValue for UPDATE operations.

Implementation*:* In SQL Server, audit triggers are implemented using AFTER INSERT, AFTER UPDATE, and AFTER DELETE trigger definitions on each sensitive table. For SELECT audit logging which SQL Server triggers do not natively support Canyon Ranch should implement SQL Server Audit objects at the database level targeting the sensitive schemas, or use Extended Events sessions to capture read access to restricted tables. The audit log table itself must be protected from modification by all roles except the DatabaseAdministrator, and audit log records should be retained for a minimum of seven years to support regulatory compliance and forensic investigation requirements.

Threat mitigated*:* Audit logging directly mitigates Risk 1 (insider access to health data without detection), Risk 4 (unauthorized access to preference data), and Risk 5 (inability to demonstrate compliance with erasure requests). It supports HIPAA's required implementation specification for audit controls under the Security Rule (HHS, 2024), CCPA's requirement to demonstrate

compliance with consumer rights requests (California Attorney General, 2024), and GDPR's accountability principle under Article 5(2) which requires that Canyon Ranch be able to demonstrate compliance with all data protection principles (European Parliament & Council of the European Union, 2016).

Recommendation 3

Implement Column-Level Encryption for PHI and PII Attributes

What it is: Column-level encryption is a database security technique in which specific sensitive columns are encrypted at rest using strong encryption algorithms, so that even a user or attacker with direct access to the database files or backup media cannot read the protected values without the appropriate decryption key (Jukic et al., 2026). In SQL Server, this can be implemented using Always Encrypted, a feature that ensures sensitive data is never exposed in plaintext to the database engine itself, protecting data even from database administrators.

Specific tables and columns protected: The highest priority columns for encryption are those in the Restricted/PHI tier: booking.QUESTIONNAIRE (AnyRecentSurgery, IsPregnant), booking.PREFERENCE (MassagePressure, TherapistGender, TimeOfDay), and service.SERVICEAPPOINTMENT (MedicalAlertFlag). In the Confidential tier, the following PII columns should also be encrypted: booking.GUEST (DateOfBirth, Email, Street, City, State, ZipCode), booking.GUEST_PHONENO (PhoneNo), and billing.GUESTFOLIO (PaymentMethod, Amount). The CENTMEMBERSHIP discount rate columns (DiscountRate, RoomDiscount, ServiceDiscount) should also be encrypted as financially sensitive business data.

Implementation*:* Canyon Ranch should implement SQL Server Always Encrypted with a Column Master Key (CMK) stored in Azure Key Vault or Windows Certificate Store, separate from the database server, so that encryption keys are never co-located with the encrypted data. Randomized encryption should be used for health and preference columns where equality searches are not required, providing stronger security against pattern analysis attacks. Deterministic encryption should be used for columns such as Email and GuestID where equality lookups are necessary for application functionality. In addition to column-level encryption, Canyon Ranch should enable Transparent Data Encryption (TDE) at the database level to encrypt the entire database at rest, providing a defense-in-depth layer that protects against physical theft of database files or backup media (Syverson & Murach, 2023).

Threat mitigated*:* Column-level encryption directly mitigates Risk 1 (unauthorized access to health data even by privileged database users), Risk 3 (insecure storage of financial and payment data), and Risk 5 (data exposure in the event of a breach affecting retained historical records). It supports HIPAA's technical safeguard requirements for encryption of PHI (HHS, 2024), CCPA/CPRA's requirement to implement reasonable security measures for sensitive personal information (CPPA, 2023), and GDPR's Article 32 requirement to implement appropriate technical measures to ensure a level of security appropriate to the risk, explicitly citing encryption as an example measure (European Parliament & Council of the European Union, 2016).

References

1. Applegate, L. M., & Piccoli, G. (2004). *Canyon Ranch* (Harvard Business School Case No. 805-027). Harvard Business School Publishing.
2. California Attorney General. (2024). *California Consumer Privacy Act regulations*. https://oag.ca.gov/privacy/ccpa
3. California Privacy Protection Agency. (2023). *California Consumer Privacy Act (CCPA) as amended by the California Privacy Rights Act (CPRA)*. https://cppa.ca.gov/regulations/
4. European Parliament & Council of the European Union. (2016). *Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation)*. *Official Journal of the European Union, L 119*, 1–88. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679
5. Federal Trade Commission. (2024). *Health breach notification rule: Final rule*. *Federal Register, 89*(84), 37172–37236. https://www.ftc.gov/legal-library/browse/rules/health-breach-notification-rule
6. U.S. Department of Health and Human Services. (2024). *HIPAA Security Rule*. https://www.hhs.gov/hipaa/for-professionals/security/index.html
7. U.S. Department of Health and Human Services. (2025). *Summary of the HIPAA Privacy Rule*. https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html
8. Jukic, N. (2026). *Database systems: Introduction to databases and data warehouses* (3rd ed.). Prospect Press.
9. Syverson, B., & Murach, J. (2023). *Murach's SQL Server 2022 for developers*. Murach Books.

Contribution Statement for Milestone 3

Contribution Statement

Suggested Role	Primary Owner	Core Responsibilities
Team Coordinator and Integration Manager	Abigail Torome (Member 3)	Led the final revision pass addressing all M2 feedback and compiled the final Revision Summary. Integrated all sections from M1, M2, and Final into a single, coherent, professionally formatted Word document. Ensured data dictionary is complete and consistently formatted across all tables. Wrote the abstract for the final submission. Manages the final database backup and submission package.
DW Conceptual Designer	Swetha Lnu (Member 4)	Designed the star or snowflake schema; selected grain for the fact table, identified additive vs. semi-additive measures, and defined all dimension tables. Identified which guest dimension attributes require SCD Type 1 vs Type 2, with written justification. Produced the DW schema diagram in ERDPlus (using distinct visual treatment for fact vs. dimension tables). Wrote the conceptual DW design narrative, including grain statement and schema choice justification.
DW Physical Developer	Isaac Wainaina (Member 5)	Created the CR_DW schema in SQL Server. Wrote CREATE TABLE statements for all fact and dimension tables with surrogate keys and the fact_/dim_ naming convention. Wrote the T-SQL script to populate dim_Date for all dates from Jan 1, 2000 to Dec 31, 2010 using a recursive CTE or WHILE loop. Created all FK constraints between the fact table and dimension tables. Attempted the Extra Credit ETL: wrote INSERT...SELECT statements to load the fact and dimension tables from the relational database.

Data Analytics Developer	Sanjay Singh (Member 1)	Wrote and tested all four DW analytical queries: ROLLUP, GROUPING SETS, time-series window function, and the custom business intelligence query. Selected one analytical requirement from Section 3.2 and clearly stated the business question and supported decision. Provided SSMS screenshots, required interpretations and plain-English business explanations for all four queries. Wrote the analytical queries section of the report.
Ethics & Privacy Analyst and Data Dictionary Author	Sharon Wamalwa (Member 2)	Wrote the complete Ethics & Privacy Analysis (all five required subsections): data sensitivity classification, five privacy risk scenarios, regulatory analysis (HIPAA, CCPA, GDPR), ethical design decisions, and three governance recommendations. Completed the Data Dictionary for all tables (column name, data type, nullable, PK/FK, domain description, sample value), formatted as tables in the Word document. Researched current regulatory frameworks to ensure the analysis reflects 2025-2026 standards.

Table 1: Team Roles and Responsibilities — Final Submission